

微网与外部电网电力调控策略

摘要

微网由光伏发电、储能设备与外部电网构成，需要在电价与光伏出力都不完全可行的条件下安排购电与充放电。本题四问共用同一套物理系统与同一条市场规则，差别只在“制定计划时能知道什么”；本文据此把四问写成一条递进主线：信息与计价规则每放宽一层，费用为何变化、变化多少，以及这个结论在参数与信息扰动下是否稳定。

问题一的信息完全已知。以 144 个十分钟时段建立含能量平衡、储能递推、功率与储电量边界、日初日末储电量相等的线性规划，两阶段词典序求解（费用优先、再压缩无效充放电）。结果为全天购电量 59,482.70 kWh、购电费 **35,126.95 元**，较不配置储能降低 **26.90%**；并由对偶标定出日末电量估值系数 **0.4720 元/kWh**，供后三问共用。

问题二引入光伏不可预见：计划只用 0:00 可得的信息，以过去 4 天光伏均值作点预报，再按历史相对误差的 20% 分位逐日下调形成保守裕度，结算用实际光伏。全年费用 **13,252,341.09 元**（紧急购电费 360,522.85 元，242 天/375 段）；与完全信息下界相差 **99.76 万元（7.53%）**，这是预报误差的不可消除代价，而相对同对象“无裕度”方案低 **92.57 万元（6.53%）**。

问题三增加四个整点预报与偏差计价（计划高于实际取用按 50%、超出按 150%、缺口按 5 倍价），并按日内更新只调整尚未执行的时段，执行时允许回退与实测冲突的计划动作。全年费用 **13,120,194.06 元**、紧急购电量 42,672.22 kWh；四个时刻预报较仅用 0:00 预报节省 **443,957.78 元（3.27%）**，执行规则再降 **24.95 万元（1.87%）**、紧急购电量下降 **55.5%**。

问题四只把电价换成逐日逐时段的波动电价，结果为 **13,850,454.64 元**与 **13,727,033.66 元**，较对应固定电价方案上升 **4.51%**与 **4.63%**。机制上，波动电价不是套利红利，而是相关性风险：日级价格水平与当日净负荷的相关系数为 **0.9824**，而日内价格形状与净负荷形状几乎不相关（0.093）；摘要引用的 0.9824 为全年 365 天窗口，本问评价期 334 天窗口为 0.9801，两者相差 0.0023，不影响机制判断。交叉结算显示，价格环境效应为 **+744,522.48 元**与 **+760,032.97 元**，策略重优化仅抵消 146,408.94 元与 153,193.38 元。

稳健性做参数、信息、机制三层单轴灵敏度：日末电量估值系数在 $\pm 20\%$ 内变化时全年费用变动不超过 **0.26%**；储能效率取 0.85、0.90、0.95 时各问费用同向变化 3.4%-4.2%；预报误差强度减半或放大 50% 时费用变化 **-2.40%**与 **+2.73%**，误差主要通过调整层传导。综上，四问的递进量化了“信息与计价规则”的价格：信息越充分、偏差计价越精细，费用越低，而可回收空间已基本收干。

关键字：微网购电；储能调度；线性规划；滚动调整；灵敏度分析

一、问题重述与分析

1.1 问题背景

微网由光伏发电、储能设备与外部电网构成，为小区负载供电 [4]。由于光伏出力与用电需求在时间上并不匹配，微网必须协调”购电”与”储能充放”两个动作：在电价较低或光伏富余时充电，在电价较高时放电，从而把购电需求从高价时段转移到低价时段。

这一协调受到三类硬性限制：储能容量 12000 kWh；单时段充电量与放电电量均不超过 833.333 kWh（对应最大充放电功率 5000 kW）；储电量在任何时刻都必须保持在 1200-10800 kWh 之间；充放电单向效率为 90%。同时，微网不允许向外部电网售电。

在物理约束之上，还有两类不确定性会改变购电计划：光伏实际出力会偏离预报，外部电价也可能逐日逐时段波动。市场规则则对”计划与实际执行的偏差”另行计价：供电缺口按交易时刻电价的 5 倍紧急购电；计划购电量高于实际取用量的部分按交易时刻电价的 50% 计价、超出部分按 150% 计价。

因此，四个问题可以在同一条主线上理解：

同一套微网物理约束、同一条市场规则，随着”制定计划时能知道什么”逐层放宽，最优购电与储能策略如何变化、费用为何变化、以及这些结论在参数与信息扰动下是否稳定。

1.2 四个问题的任务与递进关系

问题	制定计划时已知什么	决策量	偏差如何计价	需要输出
一	全部已知：电价、负载、光伏功率的当日预测（附件 1）	全天 144 个时段的购电量与充放电电量	无偏差项（计划即执行）；要求当日 0:00 与 24:00 储电量相同	指定时段购电量、充放电电量、全天购电量与购电费（result1.xlsx）
二	电价固定（附件 1）；当日负载已知；光伏不可预见（计划只能使用决策时刻可得的信息）	逐日计划购电量、充放电电量	供电缺口按 5 倍价紧急购电	指定日期按表 1、表 2 格式的结果与紧急购电量（result2.xlsx）
三	每天 0:00、6:00、12:00、18:00 均可获得未来 24 小时整点光伏预报（附件 3）；0:00 的计划建立在整点预报之上，随后三个时刻的预报用于更新	0:00 计划购电量与后续各预报时刻的调整量	计划高于实际取用按 50%、超出按 150%，缺口按 5 倍价	指定日期按表 1、表 2、表 3 格式的结果（result3.xlsx）
四	电价改为附件 4 的逐日逐时段实时价，其余信息结构与问题二、问题三分别相同	同问题二 / 同问题三	同问题二 / 同问题三	分别保存完整购电策略（result4-2.xlsx、result4-3.xlsx）

四问依次只增加一个新要素：问题一没有不确定性；问题二加入”光伏不可预见”；

问题三加入” 预报可更新” 与” 偏差计价”；问题四加入” 价格波动”。这样，任意两问之间的费用差都可以归因到某一个具体要素上，而不是笼统的” 条件不同”。

1.3 三个口径事实

建模之前有四处约定会直接影响结果，本文先用数据本身把其中三处判定清楚（第四处见 4.4 的终端处理）。

（1）附件 1 是附件 4 的年均水平，四问共用一根标尺。逐时段比对可知，附件 1 的电价等于附件 4 在全 365 天上的逐时段算术均值（保留 4 位小数），逐时段最大绝对差 5.15×10^{-5} 、相关系数 0.9999999956。附件 1 的负载列与光伏列同样是附件 2 的全年均值剖面。判据窗口必须写清：该结论只在” 全 365 天” 窗口下成立（改用 334 天评价窗或 1 月窗会得到不同的数值，不能混用）。

由此，问题一至问题三使用的” 典型日” 电价，正是问题四波动电价的年均水平。所以问题四与问题二/三之间的费用差只来自电价的时序结构，不包含” 电价水平整体变化” 的伪影。

（2）时间口径按数据排布判定，不按标签字面判定。附件与结果模板给出的 144 个时间标签是区间的右端点，与数据位置相差一格。判定依据来自数据自身：附件 2 中每日的末列与次日首列不相同（364 天中 0 天相同），且行内相邻变化（约 -0.4 kW ）比跨日跳变（数十 kW ）小两个数量级，说明 144 个值就是当日 0:00-24:00 的完整时序。因此本文约定：结果文件第 1 个数据位置对应 [0:00, 0:10)，第 144 个数据位置对应 [23:50, 24:00)；模板中可见的 0:10、…、0:00+1 标签整体比实际时段右移一格。全文（含问题一至问题四）统一采用这一位置对应关系。

（3）不确定性可分三层，四问正是逐层放宽。价格（固定 / 逐日逐时段波动）、光伏出力（不可预见 / 有预报 / 预报可更新）、负载（题面未给负荷预报）构成三个层次的输入不确定性；市场规则（5 倍紧急购电、50%/150% 偏差计价）决定这些不确定性如何转化为费用。1.2 的递进表即为这一分层的展开。

1.4 各问的分析要点

问题一：先立基准与两把标尺。电价、负载与光伏预测全部已知，可作全天联合调度。由于充放电会改变后续时段的可用电量，各时段决策通过储电量相互耦合，不能按当前供需缺口逐段独立决定，因此适合建立线性规划 [1, 2, 13, 8]。求解分两阶段：先最小化全天购电费用，再在费用锁定条件下最小化充放电总量，以消除” 同时充放” 造成的退化最优解。该问还给出后三问要用的两把标尺，储能相对” 不配置储能” 的价值，以及日末剩余电量的边际价值。

问题二：把” 不可预见” 写进计划。计划与实际执行在此分离：计划偏少会在缺口处触发 5 倍价紧急购电，计划偏多则会产生已付费而未利用的电量（题面不允许返售），

二者代价不对称，必须在“更准”与“更稳”之间取平衡。本文的处理是：用过去 4 天光伏均值作点预报，再按历史相对误差分布的分位数形成逐日更新的保守裕度；计划只使用决策时刻已获得的信息，结算使用实际光伏，缺口按 5 倍价紧急购电。

问题三：两处新增自由度。一是信息上的，预报来源由问题二的自建预测换成题面给定的整点预报（附件 3），且 6:00、12:00、18:00 的更新可以修正尚未执行时段的安排；二是执行上的，实测与计划发生冲突时，是否仍照原计划执行。同时，调整本身有代价（50%/150% 计价），因此“是否调整、调整多少”需要权衡预报收益与调整代价。特别地，执行层必须避免两类明显浪费：实测已经富余却仍按计划放电，以及为了完成“只能依靠 5 倍价紧急电才能实现”的计划充电而多买高价电。

问题四：把价格效应与策略效应分开。该问只更换价格来源，其余信息结构与问题二、问题三完全相同，因此可以用同一套策略在两套价格下交叉结算，把费用变化分解为“价格环境的影响”与“策略重新优化的作用”两部分。需要强调的是，价格波动未必带来套利红利：若高价时段恰好是必须购电的时段，波动只会推高成本。本文据此进一步考察价格与净负荷的相关性，解释储能能够缓解多少。

1.5 全文技术路线

按上述主线，本文的技术路线为：先在同一物理约束与市场规则下建立问题一的确定性基准，标定储能价值与日末电量价值；再依次引入光伏不可预见、预报更新与偏差计价、价格波动，逐问给出策略与费用；最后从参数、信息、机制三个层面做灵敏度分析，回答结论的适用范围。整体路线见图 1。

数据: 附件 1 典型日 (负荷/光伏/电价) | 附件 2 实际负荷与光伏
附件 3 光伏预报 | 附件 4 逐日 144 点实时电价
关键事实: 附件 1 = 附件 4 逐时段全年均值日 (corr≈1.000)
约束: $\eta=0.9$; SOC ∈ [1200, 10800] kWh; 充放功率 ≤ 5000 kW
(10 min ≤ 833.33 kWh)
紧急购电 $5 \times P$; 评估期 334 天 $\times$ 144 段; 终端残值 $\lambda=0.4720$ (弱参数)

确定性典型日·附件 1 价

日费用 35,126.95 元; 无储能上界 48,052.05 元 → 储能降本 12,925.10 元 (26.90%)

↓ 口径升级：确定性 → 非预见 + 因果保守裕度

334 天 × 144 段 · 附件 1 价

全年费用 13,252,341.09 元 (计划 12,891,818.24 + 紧急 360,522.85)

↓ 执行层升级: E1 → v2b, 结算升级到读法 C

读法 C 结算·执行器 v2b

全年费用 13,120,194.06 元；滚动信息价值：仅 0:00 → 13,564,151.84 元，
四时点 → 13,120,194.06 元（省 3.27%）

↓ 电价口径：附件1典型日→附件4逐日144点

附件 4 逐日 144 点波动电价

用附件 4 实际电价替换典型日价，问题二、问题三各重算一次；
再用 2×2 反事实（换价×重优化）与四时段分块把「冲击」与「调整」分开

Q4-2 = 13,850,454.64 元、Q4-3 = 13,727,033.66 元；
价格冲击 +744,522.48 元（+5.618%）→ 重优化对冲 146,408.94 元（19.7%）
→ 净 +598,113.55 元（+4.513%）

③ 机制结论（本题要点）

波动电价抬价来自「价格水平与净负荷正相关」(corr 0.9801, 334 天), 不是套利红利; 日前重优化可对冲 19.7% 的价格冲击。

图1 本文技术路线：从典型日最优计划到波动电价决策

二、模型假设与符号说明

2.1 题面给定的口径（直接采用）

假设 1（时间离散）：以 10 min 为一个调度时段，每日 144 段；时段内功率视为恒定，电量 = 功率 × 1/6 h。所有购电量、充放电量、光伏与负载电量均按此时段尺度表述。

假设 2（储能参数）：储能容量恒为 12000 kWh；单时段充、放电量各不超过 833.333 kWh（对应最大充放电功率 5000 kW）；储电量始终保持在 1200-10800 kWh 之间；充放电单向效率取 0.9（往返 0.81），即储电量按 $S_t = S_{t-1} + 0.9 C_t - D_t / 0.9$ 递推。效率按题面给定的 90% 同时作为充电效率与放电效率。

假设 3（市场规则）：微网只从外部电网购电，不允许售电；供电缺口按交易时刻电价的 5 倍紧急购电；问题三中，计划购电量高于最终实际取用量的部分按交易时刻电价的 50% 计价，超出部分按 150% 计价。

2.2 信息可得性假设（本文关键假设）

这四条决定“计划阶段能用什么数据”，是四问差异的来源，逐条给出依据与边界。

假设 4 当日负载曲线已知。问题二至问题四中，每天 0:00 可获得当日负载曲线。依据是题面只为光伏给出预报机制（附件 3），并未提供任何负载预报，而附件 2 给出逐年逐时段的负载。其边界为：若进一步假设负载也不可预知，改用过去 4 天的情景，问题二全年费用将由 13,252,341.09 元升至 16,110,845.24 元，增幅 21.6%，该假设的作用范围由此可以量化。

假设 5 光伏实际值只用于结算，预报按其发布时刻进入决策。尚未发生的实际光伏出力不参与计划制定。问题二用过去 4 天光伏均值作点预报并叠加保守裕度；问题三按 0:00、6:00、12:00、18:00 发布的整点预报重建十分钟尺度输入，且每个时刻的预报只用于修正尚未执行的时段。依据是题面给出的光伏发电预测功率与附件 3 的整点预报即为此用途；验证由独立因果性检验给出，改动当日实际光伏并不改变计划阶段的计划量。

假设 6（问题四的当日电价已知）：问题四假设每天 0:00 可获得附件 4 给定的当日电价序列，并据此制定策略；其余信息结构与对应的问题二 / 问题三保持一致。依据：题面在问题四只要求“针对波动电价重新计算问题二和问题三”，未给出任何价格预报机制（预报机制只出现在问题三且只针对光伏），因此以给定序列作为计划输入是题面最直接的读法。边界：本文另设对照，若计划改用上一周同日价格、结算仍用当日实际价格，问题四滚动策略全年费用由 13,727,033.66 元升至 13,881,852.23 元（+1.13%），见附录对照。

假设 7（预热期与跨日衔接）：2025 年 1 月为历史信息准备期（储能不充不放，1 月 1 日 0:00 的 6000 kWh 保持至 2 月 1 日）；正式评价期为 2025-02-01 至 2025-12-31 共 334

天，每日初始储电量取前一日末的实际值，直至评价期末。

2.3 建模简化（及其影响）

假设 8（结算与调用量的区分、若干次要损耗略去）

1. 同一时段内不同时充电与放电（若同时出现，只保留净额），因此充放电量与充放电功率的界都按净额判定。
2. 问题二中，普通购电按计划量全额结算，未调用部分不返售、不跨时段转移，也不进入实际能量平衡；问题三中，最终只按最终生效的调整量结算一次，中间调整版本不重复计费。
3. 光伏无法被负载或储能利用时允许弃光；除储能转换损耗外，不另计微网内部输配电损耗。
4. 忽略储能自放电、容量衰减、循环寿命损耗与额外运维成本。

影响：上述简化会低估储能的长期使用成本、也可能低估弃光带来的收益损失；但由于四问采用同一套简化，跨问比较与灵敏度结论不受影响。终端储电量的估值处理见 4.4。

三、符号说明

下标 d 为日期（评价期 334 天）， t 为日内十分钟时段（每日 144 段）， $\tau \in \{0, 6, 12, 18\}$ 为预报发布时刻。全文符号与正文、图表、附录严格一致。

符号	含义	单位
(一) 题面给定参数		
p_t	固定电价，附件 1 的典型日分时电价	元/kWh
$p_{d,t}$	逐日逐时段电价，附件 4	元/kWh
$L_{d,t}$ 、 $E_{d,t}^L$	负载功率与负载电量， $E_{d,t}^L = L_{d,t} \Delta t$	kW、kWh
$P_{d,t}^{PV}$ 、 $E_{d,t}^{PV}$	实际光伏功率与实际光伏电量	kW、kWh
$\hat{P}_{\tau}^{PV}(u)$	发布时刻 τ 对未来整点 u 的光伏预报功率	kW
η	单向充放电效率，取 0.9	—
Δt 、 T	单个时段长度 1/6 h、每日时段数 144	h、—
$C^{\max}$ 、 $S^{\min}$ 、 S^{up} 、 S^{cap}	充放电量上限 833.333、储电量下界 1200、上界 10800、容量 12000	kWh
(二) 决策变量		
G_t	问题一的外网购电量	kWh
$G_{d,t}^{pl}$	问题二与问题四对应分支的计划购电量	kWh
$G_{d,t}^0$	滚动调整模型在 0:00 制定的原始计划购电量	kWh
$A_{d,t}$ 、 $U_{d,t}$	最终生效的调整购电量、实际调用的普通购电量	kWh
$H_{d,t}$	紧急购电量	kWh
$C_{d,t}$ 、 $D_{d,t}$	储能充电量与放电量，交流侧计量	kWh
$S_{d,t}$ 、 $R_{d,t}$	时段末储电量、弃光量	kWh
(三) 派生量与评价量		
$\hat{E}_{d,t}^{PV}$	由历史均值构造的基础光伏预测电量	kWh
κ_d	问题二逐日更新的因果保守裕度，取历史相对误差的 0.2 分位	—

符号	含义	单位
α 、 μ	问题三净需求保守度分位数取 0.8、附加光伏裕度取 0	—
$r_{d,t}$ 、 $Q_{\alpha}(\cdot)$	相对预测误差、分位水平为 α 的分位数算子	—
$WG_{d,t}$	已付费但未调用的计划购电量	kWh
λ	日末库存估值系数，取 0.4720	元/kWh
$c_{d,t}$	单时段现金购电费用	元
J_{plan} 、 J_{adj} 、 J_{emg}	计划购电费、调整相关费用、紧急购电费	元
J_2 、 J_3 、 $J_{4.2}$ 、 $J_{4.3}$	问题二、问题三、问题四两支柱的全年费用	元
$V_0(s)$	问题一日末储电量固定为 6000 kWh 时、初始储电量为 s 的最小购电费用	元

注： $(x)^+ = \max(x, 0)$ ；上标 pl 表示计划量，0 表示 0:00 的原始计划量，无上标表示实际执行量。

四、统一建模合同（四问共用）

四个问题共用同一套物理系统与同一条市场规则，差别只在信息可得性与计价方式。本节先把共用部分一次写清，后文各问不再重复。

4.1 时间与口径

以 10 min 为一个调度时段，每日 144 段， $\Delta t = 1/6$ h；时段内的负载功率与光伏功率按恒定处理，电量 = 功率 $\times \Delta t$ 。下标 d 表示日期， t 表示日内时段序号。

位置与标签的对应关系（四问的结果文件与全部表格统一采用）：

结果文件的第 1 个数据位置对应 [0:00, 0:10)，第 144 个数据位置对应 [23:50, 24:00)；附件与模板中出现的 0:10、0:20、…、0:00+1 等标签是相应时段的右端点标注，比实际时段右移一格。

该对应关系由数据自身判定（每日末列与次日首列不同：364 天中 0 天相同；行内相邻变化约 -0.4 kW，比跨日跳变小两个数量级），而非依据标签字面。问题一至问题四采用同一约定，以保证四问的时段含义一致。

4.2 物理约束（四问通用）

记 $C_{d,t}$ 、 $D_{d,t}$ 为时段内充电量与放电量， $S_{d,t}$ 为时段结束时的储电量， $R_{d,t}$ 为弃光量， $E_{d,t}^L$ 、 $E_{d,t}^{PV}$ 为负载与光伏电量， $\eta = 0.9$ 。

（1）能量平衡（供、需、储能、弃电在同一时段闭合）

$$G_{d,t} + E_{d,t}^{PV} + D_{d,t} = E_{d,t}^L + C_{d,t} + R_{d,t} \quad (1)$$

其中 $G_{d,t}$ 按问题不同分别表示问题一的购电量、问题二/四-二支的计划购电量、问题三/四-三支的实际调用量与紧急购电量之和（见 5.3）。

（2）储电量递推（单向效率）

$$S_{d,t} = S_{d,t-1} + \eta \cdot C_{d,t} - D_{d,t} / \eta \quad (2)$$

往返效率为 $\eta^2 = 0.81$ ；充电量按交流侧计量。

(3) 边界与市场约束

$$\begin{aligned} 0 \leq C_{d,t} \leq 833.333, \quad 0 \leq D_{d,t} \leq 833.333, \\ 1200 \leq S_{d,t} \leq 10800, \quad 0 \leq R_{d,t} \leq E_{d,t}^{PV}, \\ G_{d,t} \geq 0 \end{aligned} \quad (3)$$

其中功率上限 5000 kW 折算到单个 1/6 h 时段即 833.333 kWh；储电量受限区间为 1200 至 10800 kWh；同一时段不同时充电与放电，若同时出现只保留净额。

(4) 跨日衔接：评价期内，前一日末的实际储电量作为次日初始状态；问题一为单日问题，要求当日 0:00 与 24:00 储电量相同 ($S_0 = S_{144} = 6000$)。

微网系统结构与四问的信息递进

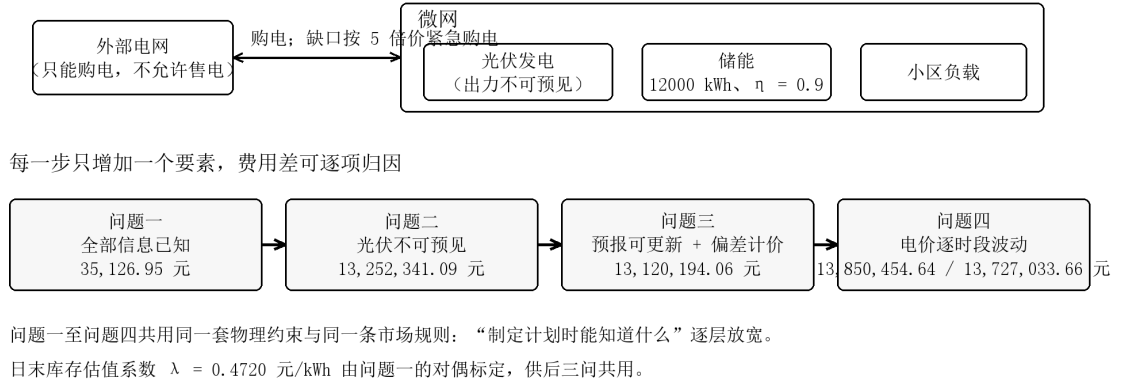


图 2 微网系统结构与四问的信息递进关系

4.3 市场规则与计价方式

四个问题共用两类市场量：计划量（决策时刻确定的购电量）与实际执行量（运行时真正取用的电量与储能动作）。计价规则按问题不同分两种结构：

(1) 计划量不可改的结构（问题二，以及问题四对应分支）

计划购电量 $G_{d,t}^{pl}$ 按计划全额结算，实际取用不足的部分按交易时刻电价的 5 倍紧急购电。单时段费用与全年费用为

$$c_{d,t} = p_t \cdot G_{d,t}^{pl} + 5 \cdot p_t \cdot H_{d,t}$$

$$J = \sum_d \sum_t c_{d,t} \quad (4)$$

其中 $H_{d,t}$ 为紧急购电量。该结构下计划量不可下调，因此“计划偏多”会直接形成已付费而未利用的电量。

(2) 计划可调整的结构（问题三，以及问题四对应分支）

设 $G_{d,t}^0$ 为 0:00 制定的计划购电量、 $A_{d,t}$ 为该时段最终生效的调整购电量、 $H_{d,t}$ 为紧急购电量。按题面，计划量高于实际取用量的部分按交易时刻电价的 50% 计价、超出部分按 150% 计价，缺口按 5 倍价计价，故单时段现金费用为

$$c_{d,t} = p_t \cdot \min(G_{d,t}^0, A_{d,t})$$

$$+ 0.5 \cdot p_t \cdot (G_{d,t}^0 - A_{d,t})^+$$

$$+ 1.5 \cdot p_t \cdot (A_{d,t} - G_{d,t}^0)^+$$

$$+ 5 \cdot p_t \cdot H_{d,t} \quad (5)$$

等价地可写成三项之和：

$$c_{d,t} = p_t G_{d,t}^0 + [-0.5 p_t (G_{d,t}^0 - A_{d,t})^+ + 1.5 p_t (A_{d,t} - G_{d,t}^0)^+] + 5 p_t H_{d,t} \quad (6)$$

三项依次为计划购电费用、调整相关费用与紧急购电费用：被取消的计划电量净退还一半费用，超出原计划的部分按 1.5 倍价计费。

即：被取消的计划电量净退还一半费用（净支出 0.5p），超出原计划的电量按 1.5p 计费。全文与结果文件中的“全天购电费”列始终等于“本表购电量 × 该时段电价”，全年总费用则由上述三项合成，不落在表格的任何一列（避免与列口径混淆）。

限度披露：题面“违约电价是交易时刻电价的 50%”一句也可按另一种方式理解，计划购电量全额照付，另按 50% 附加违约金。按该理解，全年费用为 13,972,052.33 元（附录对照）。本文采用正文口径，并把另一种理解完整列出，以便复核。

4.4 终端处理与评价期

问题一按题面要求取硬闭环 $S_0 = S_{144} = 6000 \text{ kWh}$ ，不引入任何终端残值。问题二至问题四的评价期末储电量不要求回到初始值；若只最小化现金费用，会出现“为压低费用而耗尽储能”的边界解，因此把日末储电量按其估计价值计入目标，即

$$J_d = J_{\text{cash},d} - \lambda S_{d,144} \quad (7)$$

其中 $\lambda = 0.4720$ 元/kWh 为日末库存估值系数，由问题一的对偶标定：当问题一中日末储电量固定为 6000 kWh 时，约束 $S_{144} = s_T$ 的对偶给出日末留存每提高 1 kWh 的边际成本 +0.4711 元/kWh（见 5.4）。它属于建模选择而非最优取值：灵敏度分析表明，其取值在 0.3776 至 0.5664 之间变动时，全年费用呈“平台—跳变”特征而极差仅约 0.28%（见第九章），因此本文只给出取值依据与其敏感性，不声称其最优。

评价期与预热：以 2025 年 1 月为历史信息准备期，正式评价期为 2025-02-01 至 2025-12-31 共 334 天；评价期内每日初始储电量由前一日末的实际值传递，起点为 6000 kWh。

4.5 记号约定

除第三章的符号表外，本文统一约定： $(x)^+ = \max(x, 0)$ ； $Q_\alpha(\cdot)$ 表示分位水平为 α 的分位数；上标 pl、0 分别表示“计划量”与“0:00 原始计划量”，无上标表示实际执行量。四问的可行域完全相同，差别只在决策时刻可获得的信息与计价方式。

五、问题一：确定性基准与两把标尺

5.1 模型

当日电价、负载与光伏预测功率全部已知，需求解全天 144 个时段的购电量与储能充放电量。决策变量为 G_t （购电量）、 C_t 、 D_t （充电量、放电量）、 R_t （弃光量）、 S_t （时段末储电量）。模型为

$$\begin{aligned}
 \min J1 &= \sum_{t=1}^{144} p_t \cdot G_t \\
 s.t. & G_t + E_t^{PV} + D_t = E_t^L + C_t + R_t \text{ (能量平衡)} \\
 & S_t = S_{t-1} + 0.9 \cdot C_t - D_t / 0.9 \text{ (储电量递推)} \\
 & 0 \leq G_t, 0 \leq R_t \leq E_t^{PV} \text{ (购电非负、只能弃光伏)} \\
 & 0 \leq C_t \leq 833.333, 0 \leq D_t \leq 833.333 \text{ (单时段功率界)} \\
 & 1200 \leq S_t \leq 10800 \text{ (储电量运行区间)} \\
 & S_0 = S_{144} = 6000 \text{ (题面要求日循环)}
 \end{aligned} \tag{8}$$

建立线性规划的依据：目标与全部约束对决策变量都是线性的，单目标连续优化存在精确全局最优解与可验证的对偶信息，没有必要引入启发式算法，在精确解可得时改用元启发式只会降低结论的可证性。

分两阶段求解的依据 [8]：同一最优费用通常对应多组充放电方案（例如两个相邻时段同充同放、或把一次充放电拆成两次）。为消除这类退化解中的无效能量循环，在

第一阶段得到最优费用 $J1^*$ 后，锁定费用（按数值容差）并求解第二阶段

$$\min \sum_{t=1}^{144} (C_t + D_t) \quad (9)$$

第二阶段的目的是不再优化一次费用，而是在等价最优方案中挑出储能吞吐量最小的那个，使结果可稳定复现。求解使用 HiGHS 线性规划求解器（原始/对偶可行性容差取 $1e-9$ ）；充放电互斥性由解后检查确认，必要时以混合整数模型处理。

5.2 结果

两阶段的解给出全天购电量 59,482.70 kWh、全天购电费 35,126.95 元，日初与日末储电量均为 6000 kWh，全天弃光量为 0。按题面表 1、表 2 的格式给出指定时段结果（时段对应关系见 4.1 的位置映射声明）：

表 1 问题一指定时段的购电量及全天购电量与购电费

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00—10:10	0.00	12:00—12:10	480.41	14:00—14:10	0.00
16:00—16:10	445.43	18:00—18:10	531.89	20:00—20:10	0.00
全天购电量/kWh	59,482.70	全天购电费/元	35,126.95		

表 2 问题一指定时段的充放电量及 0:00 与 24:00 储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00—4:00	4,500.00	0.00	4:00—8:00	833.33	6,365.84
8:00—12:00	4,787.96	1,703.00	12:00—16:00	5,286.04	91.10
16:00—20:00	0.00	5,780.13	20:00—24:00	5,333.33	2,859.87
0:00 储电量/kWh	6,000.00	24:00 储电量/kWh		6,000.00	

表注：四小时时段内充电量与放电量可能同时为正，表示该时段内不同十分钟区间分别发生充电与放电，并非同一时刻同时充放。

储能的价值有多大：若不允许储能（全天按缺口直接购电），购电费为 48,052.05 元；配置储能后降至 35,126.95 元，节省 12,925.10 元，降幅 26.90%。这来自时段间的价格差 [27, 14]：充电量的加权平均电价为 0.4930 元/kWh，放电量为 1.1376 元/kWh，比值约 2.31，即储能承担了“低价买入、高价替代”的转移。数据画像支持这一机制：当日电价最低 0.3713 元/kWh（05:40）、最高 1.3952 元/kWh（20:40），峰谷比 3.76；光伏富余电量合计 6,247.96 kWh，而晚峰（18:00-21:00）净负荷达 12,666.37 kWh，其中 4,026.37 kWh 即使在晚峰光伏全部就地消纳的条件下也必须由外网购电或储能放电承担。

12:00 出现 480.41 kWh 购电的解释：12:00 前后是午间电价低谷，此时光伏虽有富余，但模型仍选择少量购电给储能充电，以便在晚峰替代高价购电。这也解释了为什么

表 1 中 10:00 与 14:00 为 0 而 12:00 不为 0，表中的每个数都由”该时段在位置映射下对应的电价与净负荷”决定，而不是按标签字面读取。

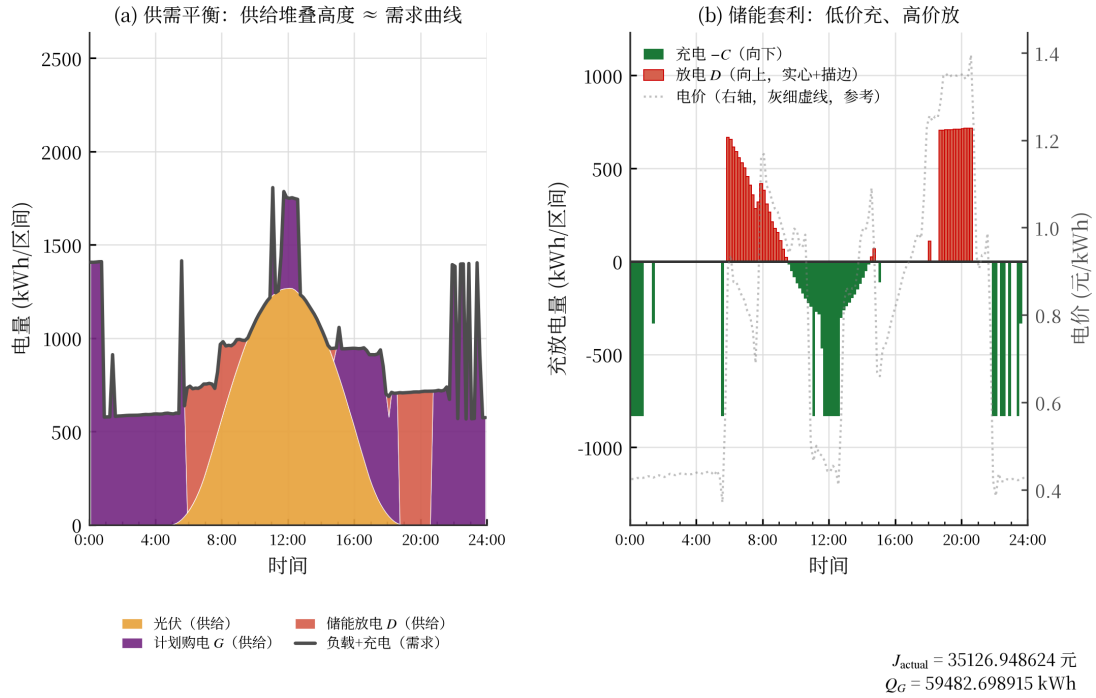


图 3 问题一供需平衡与储能充放电调度

5.3 验证

1. 独立验证器：仅用解向量与原始数据重算能量平衡、储电量递推、日循环守恒、变量边界、费用锁定与充放电互斥性，残差全部为 0；
2. 退化情形测试：光伏置零、取消储能、容量置零、效率 100% 四组极端情形均得到可解释结果（取消储能时全天购电量与 48,052.05 元基准一致）；
3. 参数扰动与口径对照：效率、容量、功率、光伏、负载、电价共 22 组扰动全部求得最优解、边界检查通过；单日问题不含跨日耦合，两种时间标签读法给出相同的全天购电量与购电费，故口径一致性以全天量保证、落位一致性以 5.1 的位置映射声明与读回检查保证。

本问为线性规划，已取得精确最优解，故不报告启发式间隙；对照基线取”不配置储能”而非更差的启发式解。

5.4 储能状态价值与日末电量估值

后三问都是滚动决策，评价期末的储电量不必回到初值，因此必须回答”日末剩余的 1 kWh 值多少钱”。问题一刚好能给出这个数：把日末储电量固定为 6000 kWh，记初始储电量为 s 时的最小购电费为 $V_0(s)$ ，则

- 约束 $S_0 = s_0$ 的对偶为 -0.4754 元/kWh，即初始储电量每增加 1 kWh，当日费用下降 0.4754 元，故初始库存的边际价值为 $+0.4754$ 元/kWh；
- 约束 $S_{144} = s_T$ 的对偶为 $+0.4711$ 元/kWh，即日末留存要求每提高 1 kWh 的边际成本；
- 两者之差 0.00433 元/kWh 是”日初与日末同向各挪 1 kWh”的单位成本。

两条对偶必须配对正确： $V_0(s)$ 以初始储电量为自变量、日末储电量固定，其斜率对应 $S_0 = s_0$ 的对偶，而不是 $S_{144} = s_T$ 的对偶。这一点由三条互不共享代码的路径共同确认：

路径	结果（元/kWh）
线性规划解析对偶	初始端 -0.4754444444 ；日末端 $+0.4711111111$
中心差分（把右端项 ± 1 kWh 各解一次）	-0.4754444444 （差 3.2×10^{-13} ）； $+0.4711111111$ （差 2.2×10^{-12} ）
离散动态规划斜率 $-dV_0/ds$	$s = 6000$ 0.4754444444 （网格 $\delta = 2 / 1 / 0.5$ 三档一致）

配对依据是量级对比：动态规划斜率接到初始端对偶的偏差为 0，接到日末端对偶的偏差为 0.00433 （相差三个数量级）。

精度必须分层报告：离散动态规划的直接解相对第一阶段最优值 J_1^* 的残差为 6.543×10^{-5} ，经 Richardson 一阶外推后为 2.524×10^{-6} （收敛比 2.057 与 1.992，一阶收敛成立）。因此本文的表述是”离散动态规划在约 2.5×10^{-6} 量级内与线性规划互证”；该值本身是线性规划最优值的上界近似（ $V_0(6000) = 35,129.24691110$ ，比 J_1^* 高 2.298 元）。

Q1 状态价值层： $V_0(s)$ 与边际价值 $-dV_0/ds$

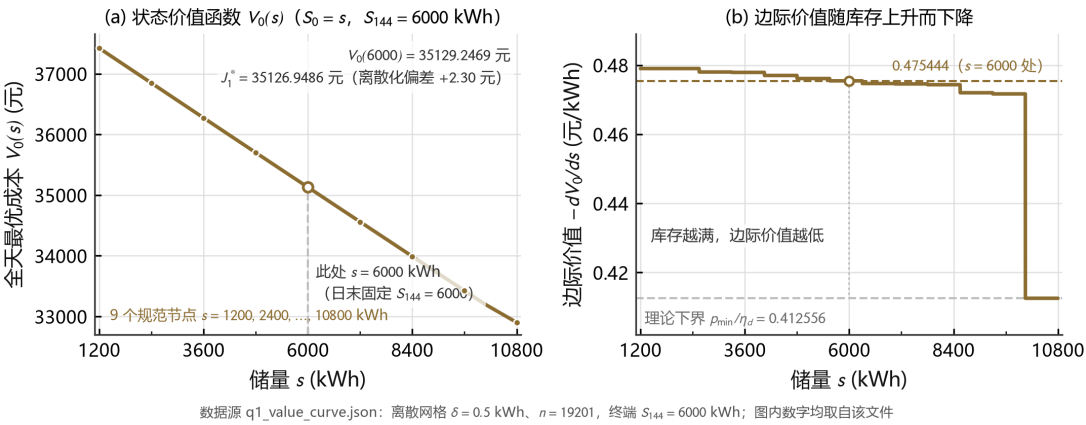


图 4 初始储电量对应的费用函数与边际价值

5.5 本问给后三问留下的两把标尺

1. 储能的价值尺度：相对不配置储能，储能在典型日带来 12,925.10 元（26.90%）的节省。后文凡涉及”储能/执行规则带来多少收益”，都以这一尺度衡量，避免把小比例收益写成重大改进。

2. 日末电量的估值尺度： $\lambda = 0.4720$ 元/kWh 由上述日末边际成本（+0.4711）标定，用于问题二至问题四的目标函数。它属于建模选择而非最优值：其影响已在第九章的参数灵敏度中检验（取值在 0.3776-0.5664 之间变动时全年费用呈”平台—跳变”特征、极差约 0.28%）。

需要说明的是：0.4754 与 0.4711 是确定性单日问题下的对偶值，后三问为滚动决策， λ 以常数估值的形式使用；本文不把确定性口径下的对偶关系直接外推到滚动问题，只把它作为取值依据并在灵敏度中检验其影响。

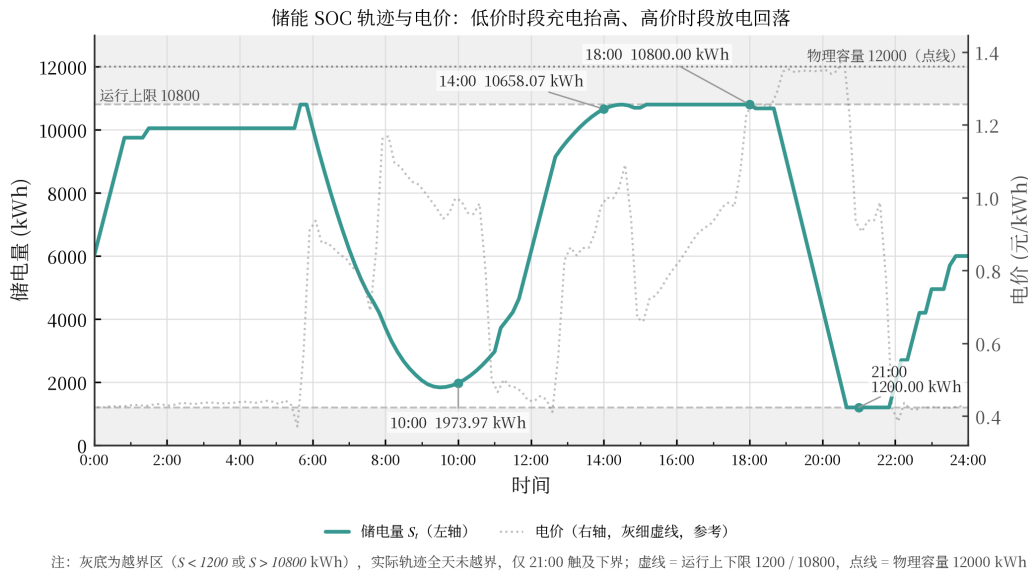


图 5 问题一储能电量轨迹与分时电价

六、问题二：不可预见的光伏与保守裕度

6.1 计划层的信息结构

问题二与问题一的差别只有一个：光伏出力不再提前已知。判断”计划阶段能否使用当天实际光伏”，不能靠直觉，也不能靠”哪个结果更省”，而要看题面给的数据是什么：

附件 1 的光伏列就是附件 2 全年逐时段光伏的均值剖面（见 1.3），即它是日前可得的预测值，不是某一天的实际值。若把当天实际光伏用于计划，紧急购电将恒为零，题面的紧急购电条款与结果表将失去意义，数学上可行，但信息结构上不成立。因此本文严格遵守：

计划阶段只使用决策时刻（当天 0:00）已经获得的信息；当天实际光伏只用于结算与执行。

这条约束不是”为了保守而保守”：它决定了后面所有结果能否被称为”可执行策略”。

6.2 预报与保守裕度

计划用的光伏预测由两步构成（均只使用 d 之前的数据）：

$$\begin{aligned}\bar{P}_{d,t} &= \frac{1}{4} \sum_{j=1}^4 PV_{d-j,t}, \\ r_{\tau,t} &= \frac{PV_{\tau,t} - \bar{P}_{\tau,t}}{\bar{P}_{\tau,t}}, \\ \kappa_d &= \text{clip}(-Q_{0.2}\{r_{\tau,t}\}, 0, 0.35), \\ \hat{P}_{d,t} &= (1 - \kappa_d) \bar{P}_{d,t}\end{aligned}\tag{10}$$

四式依次给出基础预测、相对误差、逐日保守裕度与最终预测：基础预测取最近 4 个连续自然日的同时段均值，相对误差在 d 之前的 28 天窗口内统计，裕度取下侧分位并截断在 0 至 0.35。

即：把历史”实际 ÷ 预测”的偏差取下侧分位，按该比例下调预测值，使计划偏向”少买一点”，从而降低触发 5 倍价紧急购电的概率 [7, 12]。实测裕度 m_d 的均值为 7.11%（最小 2.55%、最大 10.50%），逐日重估。

点预报窗口与误差统计窗口分别取 4 天与 28 天，与复现代码中的参数一致（ $K = 4$ 、 $WIN = 28$ ）；支撑材料另附窗口取值的对照扫描，取 4 天不改变结论方向。

分位取 20% 的依据与限度：单时段报童型条件给出 $4 \cdot \Pr(\text{实际} < \text{计划}) = \Pr(\text{实际} > \text{计划}) \Rightarrow \Pr = 0.2$ （欠购多付 4 倍差价、超购多付 1 倍）。这是启发式标定而非全模型最优，本文披露为建模选择，其影响见第九章。

6.3 计划层模型与执行规则

计划层（逐日线性规划）：决策当天 144 段的计划购电量 G_t 与充放电量，目标是使”计划购电费 + 按预测缺口计价的紧急购电风险 - 日末库存估值”最小：

$$\begin{aligned}\min & \sum_t p_t \cdot G_t + (1/K) \cdot \sum_k \sum_t 5 \cdot p_t \cdot e_{k,t} - (\lambda/K) \cdot \sum_k S_{k,T} \\ \text{s.t.} & G_t + PV_{k,t} \cdot \Delta t + D_{k,t} + e_{k,t} = L_{k,t} \cdot \Delta t + C_{k,t} + R_{k,t}^{PV} \\ & S_{k,t} = S_{k,t-1} + \eta \cdot C_{k,t} - D_{k,t} / \eta, S_{k,0} = \text{当日 0:00 储电量} \\ & 0 \leq C_{k,t}, D_{k,t} \leq 833.333, 0 \leq R_{k,t}^{PV} \leq PV_{k,t} \cdot \Delta t, e_{k,t} \geq 0 \\ & 1200 \leq S_{k,t} \leq 10800\end{aligned}\tag{11}$$

其中 k 为情景索引 [21]， $e_{k,t}$ 为情景下的缺口松弛量。本文的实现使用点预测（各情景取同一条预测曲线），因此模型退化为确定性线性规划；保留情景形式的写法是为了与”多情景方法”做同框架对照（见 6.5）。该形式下实测同时充放量约 1.7×10^{-11} kWh,

已落在物理互斥可行域内。

执行与结算规则（使用当天实际值，逐时段按净缺口处理）：记 $gap_t = L_{d,t} \cdot \Delta t - PV_{d,t} \cdot \Delta t - G_t$,

$$\begin{aligned} gap_t &= L_{d,t} \Delta t - PV_{d,t} \Delta t - G_t, \\ D_t^{ex} &= \min\{833.333, \eta(S_{t-1} - 1200), gap_t\}, \\ H_t &= \max\{gap_t - D_t^{ex}, 0\}, \\ C_t^{ex} &= \min\{833.333, (10800 - S_{t-1})/\eta, -gap_t\}, \\ S_t &= S_{t-1} + \eta C_t^{ex} - D_t^{ex} / \eta \end{aligned} \quad (12)$$

该规则是”前向执行”：先按计划量取电，再按实际缺口处置。它的两处浪费（实测已富余仍按计划放电、为完成计划充电而多买 5 倍价电）在问题三中由改进的执行规则消除（见 7.3）。

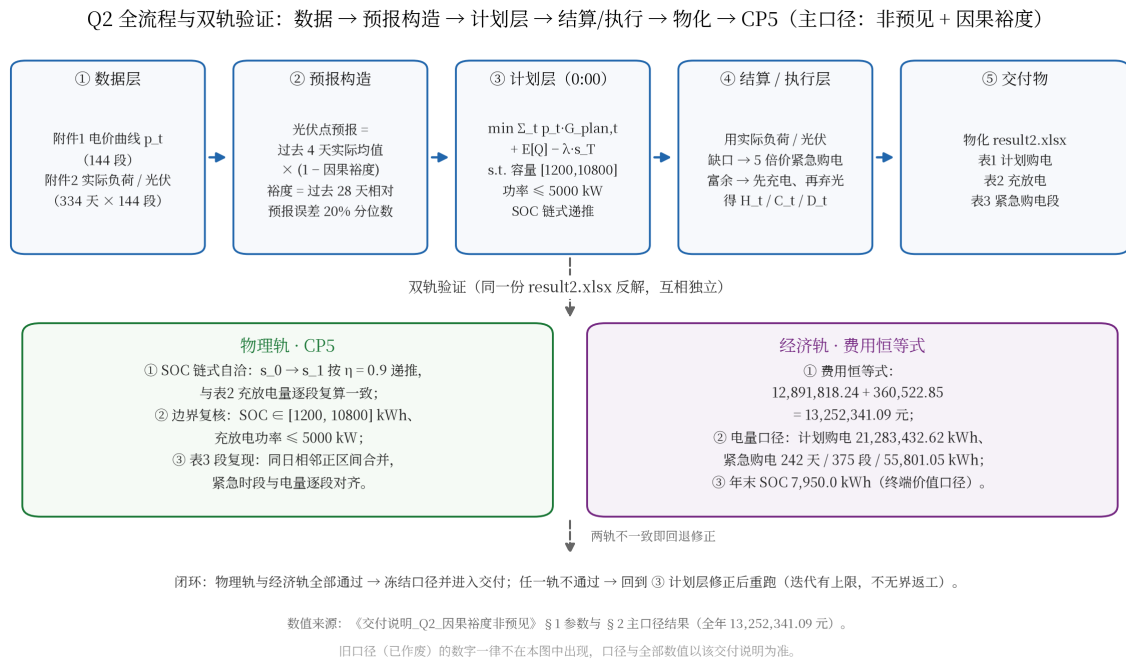


图 6 问题二日前计划与实际执行流程

6.4 结果

全年（2025-02-01 至 12-31，共 334 天）购电费用为 13,252,341.09 元，其中计划购电费 12,891,818.24 元、紧急购电费 360,522.85 元。全年计划购电量 21,283,432.62 kWh；紧急购电发生在 242 天、375 个时段，合计 55,801.05 kWh，占全年购电量约 0.26%，但因其单价为 5 倍，费用占比达 2.72%。

四个指定日期的结果（表 1、表 2 格式，时段按第 5.1 节的位置映射）：

表 3 问题二指定日期的计划购电量、全天购电量与购电费（按题面表 1 格式）

日期	时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
2025/3/20	10:00—10:10	0.00	12:00—12:10	552.31	14:00—14:10	0.00
	14:00—14:10	0.00	16:00—16:10	519.50	18:00—18:10	698.00
	全天计划购电量	67,083.45	全天购电费（当日合计）/元	41,595.10	其中计划/紧急	40,498.51 / 1,096.59
2025/6/21	10:00—10:10	0.00	12:00—12:10	0.00	14:00—14:10	0.00
	14:00—14:10	0.00	16:00—16:10	185.94	18:00—18:10	0.00
	全天计划购电量	34,795.81	全天购电费（当日合计）/元	19,334.88	其中计划/紧急	19,334.88 / 0.00
2025/9/23	10:00—10:10	0.00	12:00—12:10	555.11	14:00—14:10	0.00
	14:00—14:10	0.00	16:00—16:10	623.07	18:00—18:10	764.33
	全天计划购电量	70,722.56	全天购电费（当日合计）/元	44,095.97	其中计划/紧急	43,914.08 / 181.89
2025/12/21	10:00—10:10	0.00	12:00—12:10	1,002.77	14:00—14:10	0.00
	14:00—14:10	0.00	16:00—16:10	806.43	18:00—18:10	715.41
	全天计划购电量	95,876.74	全天购电费（当日合计）/元	61,446.07	其中计划/紧急	61,446.07 / 0.00

表 5 问题二指定日期发生紧急购电的时段与电量（按题面表 3 格式）

注：表中“全天购电费（当日合计）”为该日全部现金费用（计划购电费 + 紧急购电费），与结果文件“全天购电费”列的口径不同（后者只含计划购电量对应的费用）。

日期	紧急购电时间段	紧急购电量/kWh
2025/3/20	20:30—20:40	157.19
2025/9/23	20:30—20:40	26.07

说明：四个指定日期中只有 3 月 20 日与 9 月 23 日发生紧急购电，且当日各仅一段，故上表即为当日全部紧急购电；其对应紧急购电费分别为 1,096.59 元与 181.89 元（即当日“全天购电费（当日合计）”中“其中：紧急购电费”一栏）。2025/6/21 与 2025/12/21 当天未发生紧急购电。四日期的完整 6 时段购电量与充放电量见附录 A。

6.5 对照臂与不可预报代价

单看一个年度费用无法判断“做得好不好”。为此本文在同一数据、同一储能、同一结算规则下比较不同信息假设下的结果（完整阶梯表见附录 A）：

口径	全年费用/元	说明
计划即实际（完全信息）	12,254,696.55	不可执行，仅作理论下界
本文方案（均值预报 + 因果保守裕度）	13,252,341.09	主结果
均值点预报（不加裕度）	14,178,004.00	同对象对照：裕度价值 92.57 万元（6.53%）

由此得到两条结论：其一，预报误差存在不可消除的代价，本文方案与完全信息下界相差 99.76 万元（7.53%），该差距来自”当天光伏在 0:00 尚未发生”这一信息事实，无法靠调参消除；其二，保守裕度有效但并非免费，相对同对象的”均值点预报、无裕度”（14,178,004.00 元），裕度把费用压低 92.57 万元（6.53%），而放弃样本内最优（仅省 0.34%）是为了避免”挑配置”的质疑。本文选择的是可辩护的最优点，而不是最便宜的点。

日末电量的估值系数 λ 在 0.3776-0.5664 之间变动时，全年费用呈”平台—跳变”特征： $\lambda \leq 0.40$ 时年末储能放空至 1,200 kWh（13,251,225.13 元）， $\lambda = 0.4720$ 时年末为 7,950 kWh（13,252,341.09 元）， $\lambda \geq 0.55$ 时年末充满至 10,800 kWh（13,287,931.82 元），极差 36,706.69 元（0.277%）。可见结论对终端处理不敏感；本文取 0.4720 属于稳健区间内的建模选择（来源见 6.4）。

6.6 验证

1. 独立复算：用自写线性规划从官方附件独立求解问题一，得 35,126.95 元 / 59,482.70 kWh，与结果一致，验证数据链与求解器；
2. 从成品反解：仅依据交付文件中的计划购电量单元格执行本节规则，复现 375 个紧急购电时段、55,801.05 kWh 与全年费用 13,252,341.09 元，储电量链偏差约 5×10^{-6} ；
3. 因果性检验（一票否决项）：改动当日实际光伏后计划量不变，证明计划阶段未使用未来信息；
4. 逐项核查：32 项对外声明逐条设期望值与证据路径，全部通过。

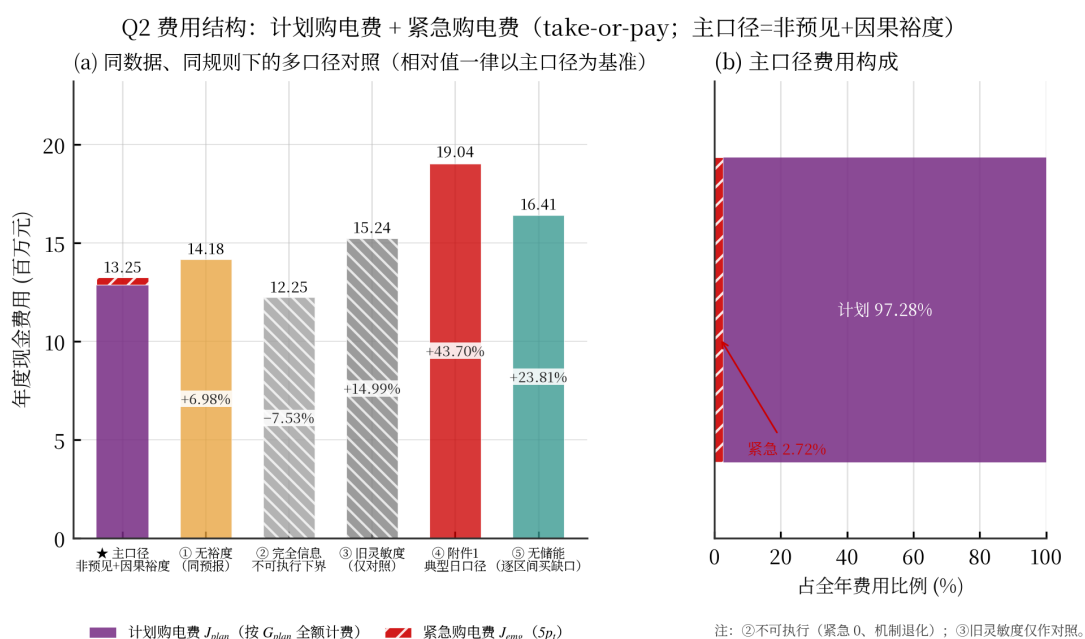


图 7 问题二评价期购电费用结构与方案对照

6.7 与下一问的衔接

问题一在”全部信息已知”下给出基准，问题二把信息放宽为”光伏不可预见”。本节在同一物理系统与同一条市场规则下再放宽一层：附件 3 的日内预报不是一次性给定，而是在 0:00、6:00、12:00、18:00 四个整点被更新；同时，实际取用量与计划量的偏差被单独计价。于是回答三件事：计划层在什么信息下制定、如何随预报更新；多引入一次预报究竟值多少钱、哪一次最值；执行层与实测冲突时还剩多少自由度。

七、问题三：预报可更新、偏差计价下的滚动调控

7.1 信息结构与计划口径

预报源为附件 3 的四个整点预报，经重构落到十分钟尺度（每日 144 槽）。规则上只允许向前看 [26, 24]：每个时刻的预报只用于修正尚未执行的时段，已执行时段固化，不用后验信息重写历史动作。当日用户负荷视为已知，与问题二同假设。储能容量区间 1200-10800 kWh、起点储电量 6000 kWh、充放电效率与功率上限沿用问题一、问题二，日与日之间储电量状态连续传递。

7.2 计价规则与费用口径

计划购电量记为 G^0 、最终生效的调整购电量记为 A 、紧急购电量记为 H ，电价为 p 。按题面：计划量高于实际取用的部分按交易时刻电价的 50% 计价、超出部分按 150% 计价、缺口按 5 倍价计价，即单时段现金费用为

$$c = p \min(G^0, A) + 0.5 p (G^0 - A)^+ + 1.5 p (A - G^0)^+ + 5 p H_{d,t} \quad (13)$$

等价地可写成三项之和：计划购电费用 $p \cdot G^0$ + 调整相关费用 $[-0.5 \cdot p \cdot (G^0 - A)^+ + 1.5 \cdot p \cdot (A - G^0)^+]$ + 紧急购电费用 $5 \cdot p \cdot H$ （被取消的计划电量净退还一半，即净支出 $0.5p$ ）。

限度披露：若把规则理解为”计划量全额照付、超用部分另加 50% 违约金”，同一物理方案全年为 13,972,052.33 元。该理解与本节的费用口径不同，仅作附录对照，不参与结果讨论。

7.3 模型与求解

计划层按日建立线性规划，以十分钟为步长、以计划购电量与储能充放电为决策量；对预报净需求取 0.8 分位作为保守取值、附加裕度取 0，以降低触发高价紧急购电的概率。滚动更新在每个预报时刻执行一次：用最新预报重新求解尚未执行的时段，已执行

时段作为既定值保留。执行层采用允许回退的实测执行规则：当实测与计划冲突时，放弃那些“只有依靠 5 倍价紧急购电才能完成”的计划动作（典型如计划充电），使执行序列在物理上可完成。评价期为 2025-02-01 至 12-31 共 334 天，与交付件同源。

7.3 补充说明：为什么问题三不沿用问题二的裕度口径

问题二只对光伏不确定做单侧保守削减，取历史相对误差的 0.2 分位；问题三的计划层直接以净需求的 0.8 分位取值，且不再叠加额外裕度。原因在于问题三的偏差计价已把“计划与调整的偏离”显式定价：计划高于实际取用按 50% 计价、超出按 150% 计价，若再叠加一层保守裕度，会造成二次保守并抬高基础购电费用。两者的目标相同，都是在紧急购电成本与计划购电成本之间取平衡，只是可用的信息结构与计价规则不同。

7.4 结果

全年费用 13,120,194.06 元，由三项构成：计划基础 12,381,239.55 + 调整相关 542,052.30 + 紧急购电 196,902.21；同期紧急购电量合计 42,672.2 kWh。以上四项均为 334 个仿真日的全年累加值。

四个指定日期（3.20、6.21、9.23、12.21）的单日全天（0:00-24:00，144 槽）指标为：计划购电量 70,918.39、34,499.06、73,703.11、96,785.58 kWh；调整购电量 65,979.69、34,173.10、67,644.50、94,515.18 kWh；当日结算费用 43,711.20、18,955.17、44,537.63、62,175.13 元。

预报更新的价值由四组方案给出：只更新 0:00 时全年 13,564,151.84 元；追加 6:00 后 13,446,385.03 元；再追加 12:00 后 13,125,320.04 元；四个时点全用为 13,120,194.06 元。相对只更新 0:00，合计节省 443,957.78 元（3.27%），全年紧急购电量由 98,295.60 kWh 降至 42,672.2 kWh（-56.6%）。单次更新的边际节省排序为 12:00（-321,065 元）> 6:00（-117,767 元）> 18:00（-5,126 元）。

执行层的自由度同样有代价可算：改用不允许回退的前向执行规则、其余口径不变时，全年费用为 13,369,682.34 元，紧急购电量 95,806.02 kWh；允许回退后分别降至 13,120,194.06 元与 42,672.2 kWh，即 -249,488 元（-1.87%）与 -55.5%。

7.5 结果的机制解释

多一次预报的价值来自它修正的时段长度与误差幅度。12:00 的更新增量最大（-321,065 元），因为它修正的是午间光伏峰值段的形状误差，而这一天段误差对计划购电量与缺口的影响最直接；6:00 次之（-117,767 元），它确定当日光伏起始段；18:00 的增量只有 0.5 万元量级，因为此时剩余时段短、储能与购电的可调整空间都被压缩。

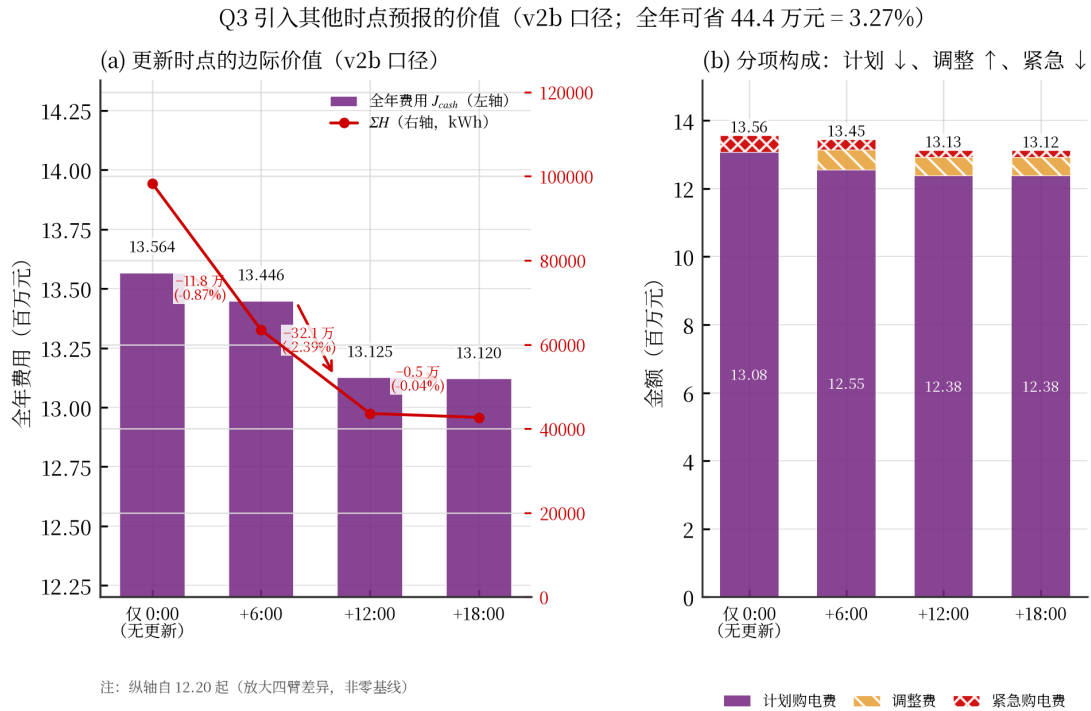


图 8 不同预报更新时刻的费用变化与边际贡献

价值的主要来源是减少 5 倍价紧急购电，而不是基础电价差：四组方案的全年计划基础项变化很小，紧急购电量却下降过半。执行规则的收益来源相同，被放弃的是那些注定要靠 5 倍价紧急电才能补回的计划充电动作。

作为对照，若假设全天光伏完全预知，可得到一个完全信息下界；该下界违反因果性、不可执行，仅作下界参照，不与本节的执行结果并列相减。

表 6 问题三指定日期的计划购电量与调整购电量（按题面表 1 格式，网格表）

日期	时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
2025/3/20	10:00—10:10	0.00	12:00—12:10	621.87	14:00—14:10	0.00
	14:00—14:10	0.00	16:00—16:10	692.13	18:00—18:10	698.00
	全天合计	2,012.00				
2025/6/21	10:00—10:10	0.00	12:00—12:10	0.00	14:00—14:10	0.00
	14:00—14:10	0.00	16:00—16:10	156.76	18:00—18:10	0.00
	全天合计	156.76				
2025/9/23	10:00—10:10	0.00	12:00—12:10	452.48	14:00—14:10	0.00
	14:00—14:10	0.00	16:00—16:10	694.96	18:00—18:10	792.73
	全天合计	1,940.16				
2025/12/21	10:00—10:10	0.00	12:00—12:10	1,069.53	14:00—14:10	0.00
	14:00—14:10	0.00	16:00—16:10	839.91	18:00—18:10	715.41
	全天合计	2,624.86				

（下表同格式给出调整购电量）

日期	时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
2025/3/20	10:00—10:10	0.00	12:00—12:10	491.43	14:00—14:10	0.00
	14:00—14:10	0.00	16:00—16:10	543.53	18:00—18:10	698.00
	全天合计	1,732.96				
2025/6/21	10:00—10:10	0.00	12:00—12:10	0.00	14:00—14:10	0.00
	14:00—14:10	0.00	16:00—16:10	102.64	18:00—18:10	0.00
	全天合计	102.64				
2025/9/23	10:00—10:10	0.00	12:00—12:10	0.00	14:00—14:10	0.00
	14:00—14:10	0.00	16:00—16:10	597.11	18:00—18:10	793.54
	全天合计	1,390.64				
2025/12/21	10:00—10:10	0.00	12:00—12:10	878.64	14:00—14:10	0.00
	14:00—14:10	0.00	16:00—16:10	839.91	18:00—18:10	715.41
	全天合计	2,433.97				

表 8 问题三指定日期的紧急购电（按题面表 3 格式，四日期作列，网格表）

2025/3/20		2025/6/21		2025/9/23		2025/12/21	
时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
8:50-9:20	151.22	—	0.00	14:20-14:50	32.36	9:00-9:20	4.67
9:30-10:00	77.61	—	0.00	17:30-17:50	5.45	9:30-10:50	108.27
14:10-14:20	6.51	—	0.00	—	0.00	—	0.00
15:30-15:40	0.72	—	0.00	—	0.00	—	0.00
15:50-16:20	33.78	—	0.00	—	0.00	—	0.00

7.6 验证

1. 独立复算：由交付件四张表逐槽重算购电量与费用，与三项分解 13,120,194.06 元逐位比对，判据为残差小于 0.01 元，实测逐位一致。
2. 因果性检验：只改动当日实际光伏、不改预报，重解后当日及此后的计划序列不变，判据为计划购电量序列逐槽差为 0。
3. 成品反解：把交付件的计划、调整、充放电、紧急购电四表联立，反解出的逐槽费用与全年费用一致，且每个时段的偏差档位与计价规则逐条对应。
4. 物理链检查：逐段功率平衡残差判据取 1×10^{-9} kWh（实测最大 3.4×10^{-13} ），储电量全程落在 1200-10800 kWh 区间内，充放电功率不越上限。

预报误差强度 ρ 减半或放大 50%，全年费用相应为 12,805,045.44 元与 13,478,837.67 元（-2.40% 与 +2.73%），而计划基础项仅变化 -0.17% 至 +0.35%，说明误差主要经由调整与紧急购电两处传导，与本节机制解释一致。

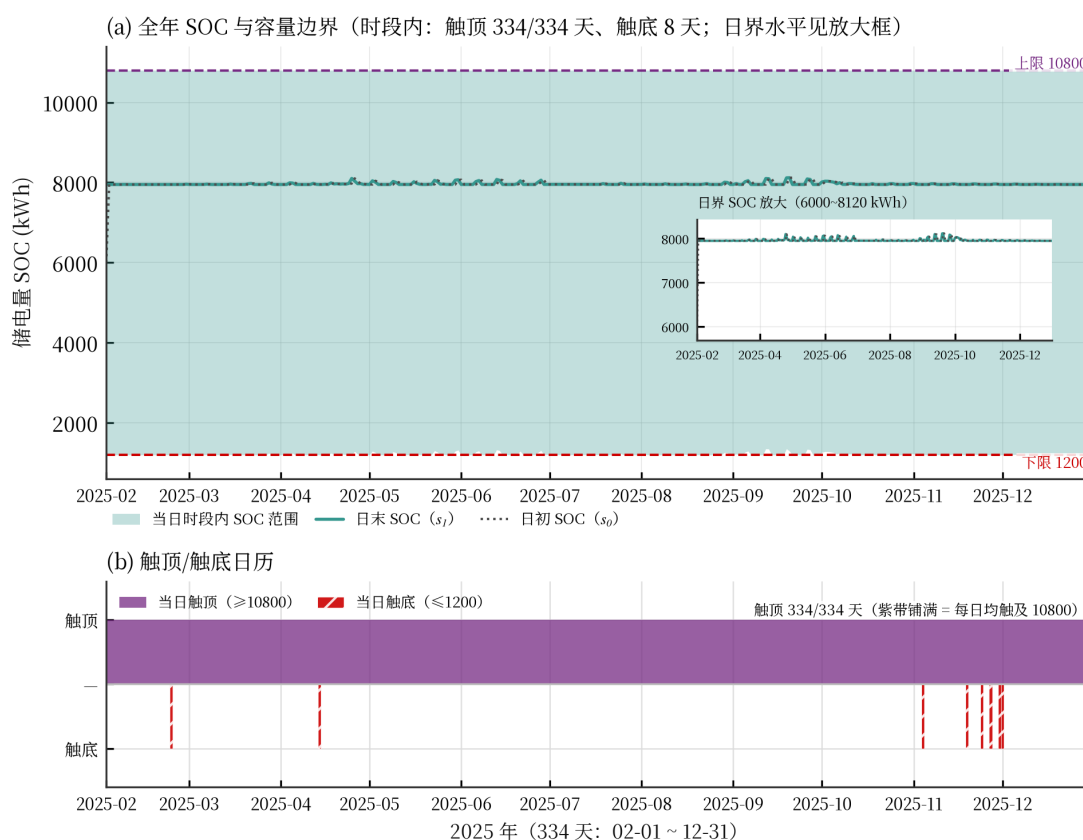


图 9 问题三评价期储电量轨迹与运行边界

八、问题四：波动电价下的重算与归因

8.1 设定：只换价格，其余一律不动

问题四不引入新模型：把电价由附件 1 的典型日曲线换成附件 4 的逐日逐时段实时电价，其余信息结构、储能参数、市场规则与执行规则分别与问题二、问题三完全一致，得到两条分支的全年结果：

分支	对应问题	全年费用/元	相对对应固定电价方案
计划不可改（同问题二结构）	问题二	13,850,454.64	+598,113.55（+4.51%）
计划可调整（同问题三结构）	问题三	13,727,033.66	+606,839.60（+4.63%）

两条分支的费用上升幅度接近（4.5% 量级），说明该上升不是某一种策略结构造成的。由于第 1.3 节已确认附件 1 等于附件 4 在全 365 天上的逐时段均值，上述差额只来自电价的时序结构，不含“电价水平整体变化”的伪影。

价格可得性说明：题面在问题四未给出任何价格预报机制，故本文以附件 4 给定序列作为计划输入（见 2.2 中的假设 6）；若计划改用上一周同日价格、结算仍用当日实际价格，滚动分支的费用由 13,727,033.66 元升至 13,881,852.23 元（+1.13%），见附录对照。

8.2 价格环境效应与策略重优化效应的分解

只报一个总数无法判断”策略到底起了什么作用”。为此本文用同一执行规则（执行只依赖计划量与实际值，与价格无关）做交叉结算：分别用两套价格生成计划，再把两套计划分别放到两套价格下结算，得到 2×2 格值（以问题三口径为例）：

计划来源\结算价格	附件 1 价（典型日）	附件 4 价（实时）
附件 1 价下优化	13,120,194.06	13,880,227.04
附件 4 价下优化	13,225,150.38	13,727,033.66

由此分解出两种效应：

效应	问题四分支二/元	问题四分支一/元	含义
价格环境效应	+760,032.97	+744,522.48	同一计划在新价格下结算的代价
策略重优化效应	-153,193.38	-146,408.94	在新价格下重算计划的抵消量
合计	+606,839.60	+598,113.55	即相对对应固定电价方案的上升

分解按未舍入值计算、残差为 0。两条结构（计划不可改 / 计划可调整）给出同向、同量级的分解，说明该结论可迁移，而非单一模型的特例。需要强调的是：价格环境效应约为重优化抵消能力的 5 倍，在本文的储能规模与信息条件下，重新优化的作用有限。

8.3 波动电价的相关性风险机制

直觉上” 电价波动 ⇒ 低买高卖 ⇒ 成本下降”，但数据给出相反结论。原因可以从三个层次看清楚：

观察	数值	含义
价格水平与当日净负荷的相关（全年 365 天窗：本问 334 天窗为 0.9801）	0.9824	高价日恰是需要多买的日
日内价格形状与日内净负荷形状的相关	0.093	日内错位空间很小
储能可调电量 ÷ 日均净负荷	9,600 / 53,710 ≈ 17.9%	储能能挪动的量级不足以对冲价格水平上移

再把” 无储能” 与” 有储能” 两种口径分别放在两套价格下比较：

环节	附件 1 价	附件 4 价	变化
无储能、逐段买缺口	16,407,319.63	17,026,782.86	+3.78%
储能的价值（无储能 - 优化结果）	4,152,623.08	4,185,349.10	+0.79%

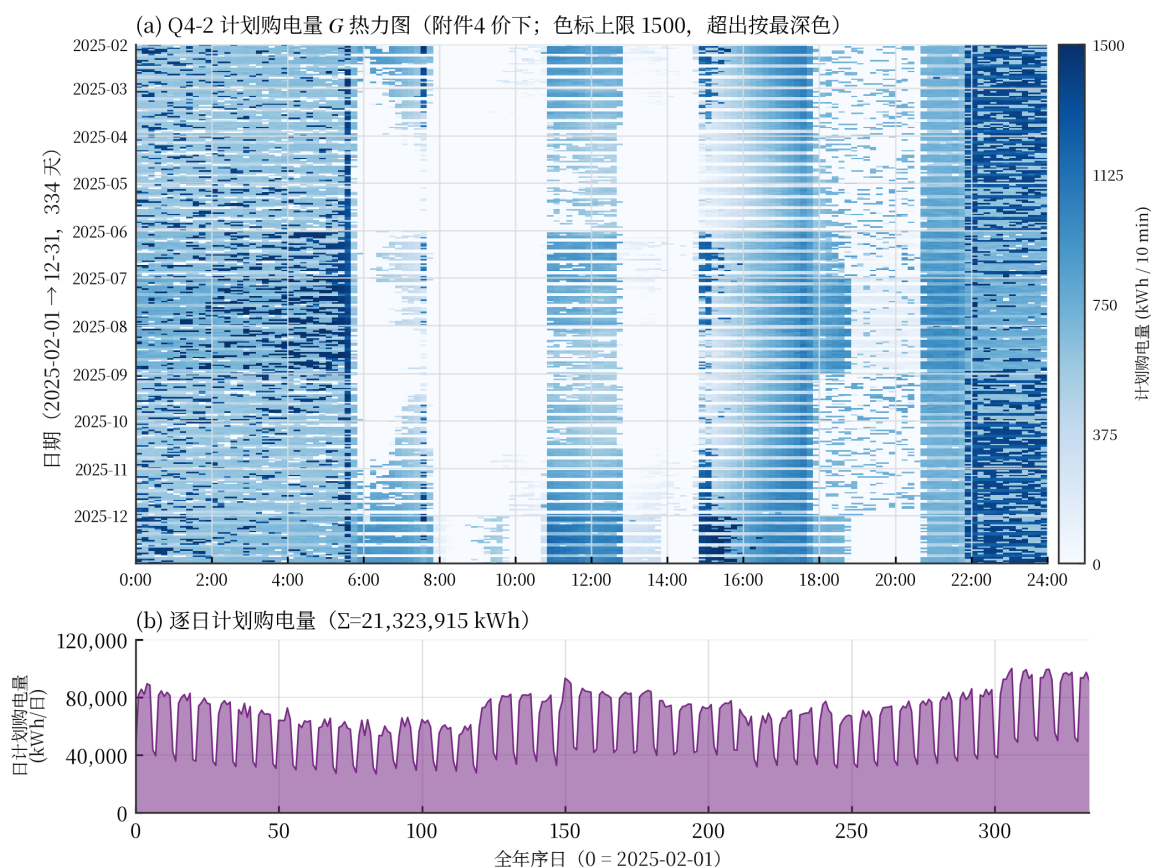


图 10 波动电价下 Q4-2 计划购电量的日内分布与逐日合计

即：价格波动几乎没有改变”储能本身的价值”，改变的是”没有储能时的代价”。因此波动电价下的响应不是”少买”，而是把购电从贵段挪到便宜段：分块看问题四分支一，夜间多买 +251,396 元、晚峰少买 -307,124 元、午间 -89,463 元，净 -146,409 元，正好等于上表中的重优化效应。

8.4 结果与图表

四指定日期 (3.20 / 6.21 / 9.23 / 12.21) 的逐日明细见附录 A；价格时序特征、计划量热力图与代表日期储电量轨迹见图 13、图 9 与图 10。

8.5 验证

1. 延拓一致性检查：把价格换回附件 1 后，两类分支分别回到 13,252,341.09 元与 13,120,194.06 元，与问题二、问题三结果一致（分支二恢复后与问题三逐位相同），说明问题四确实只换了价格；
2. 交叉结算闭合：2×2 格值与两套效应分解均按未舍入值闭合，残差为 0；反向对照（旧价格下重优化 +104,956.32 元、新计划的价格环境 +501,883.28 元）表明价格环境效应依赖计划来源，属机制诊断，不能读作”改进”；

Q4-2 代表 7 天：SOC 轨迹（实线，左轴；由 C/D 与 $\eta=0.9$ 重建）与附件4 实时电价（点线，右轴）

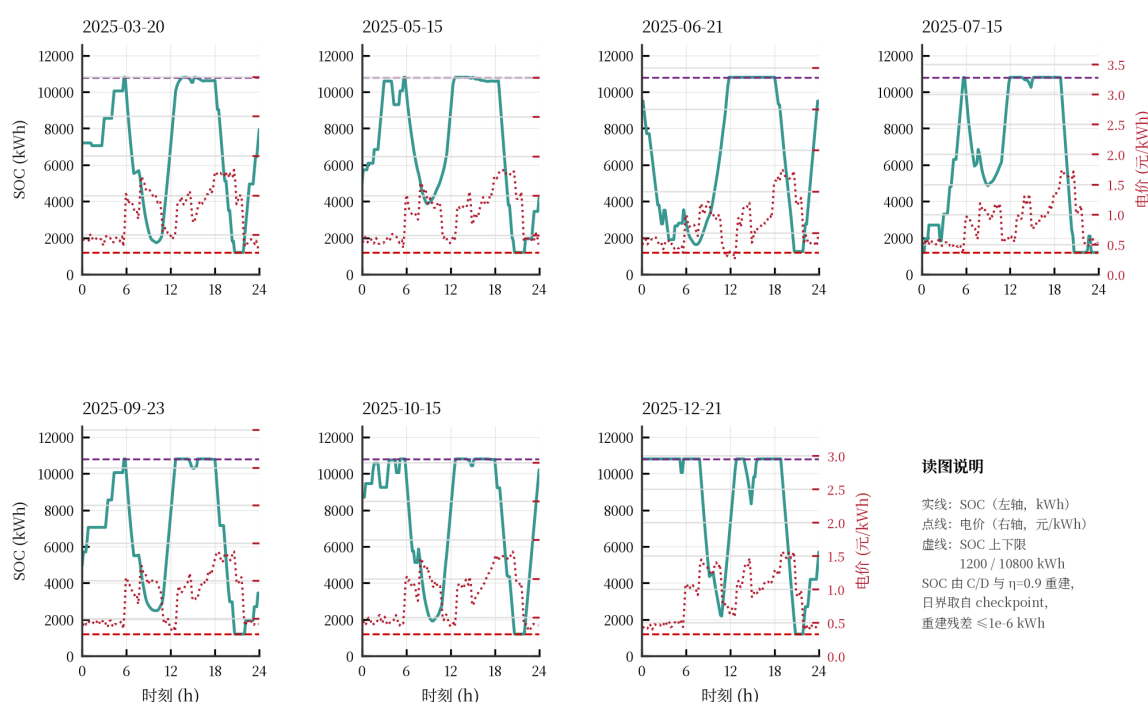
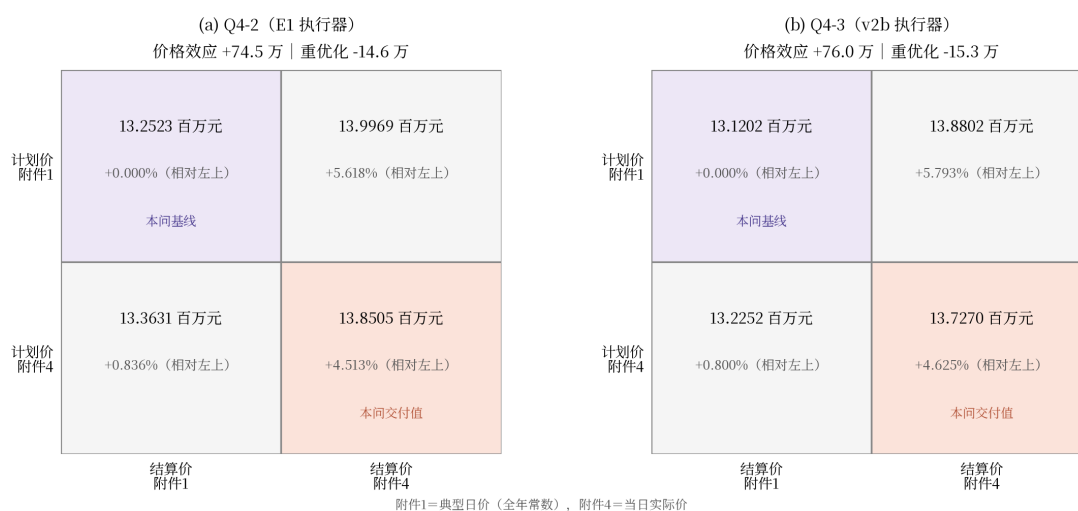


图 11 波动电价下代表日期的储电量与电价变化

Q4 价格区制反事实 2×2：左上=本问基线，右下=本问交付值



两组基线分属不同问：(a) Q2 主口径 13,252,341.09 元，(b) Q3 v2 13,120,194.06 元；组内可比，格值不可跨组横向比较。

图 12 价格环境变化与策略重优化的交叉结算对照

- 多种价格口径对照：除主口径外，另测”用上一周同日价格做计划””用附件 1 典型日曲线做计划”等口径（13,881,852.23 元、13,996,208.12 元等，见附录 A），结论方向不变；
- 主动止损：另测十条候选改进轴（价格分块、裕度结构、终端处理等），全部落在 0.5% 量级以内，按预注册的 0.5% 止损线封存、不进入方案，其含义是”在本文的储能规模与信息条件下，这些方向的边际收益不足以支撑更复杂的策略”。

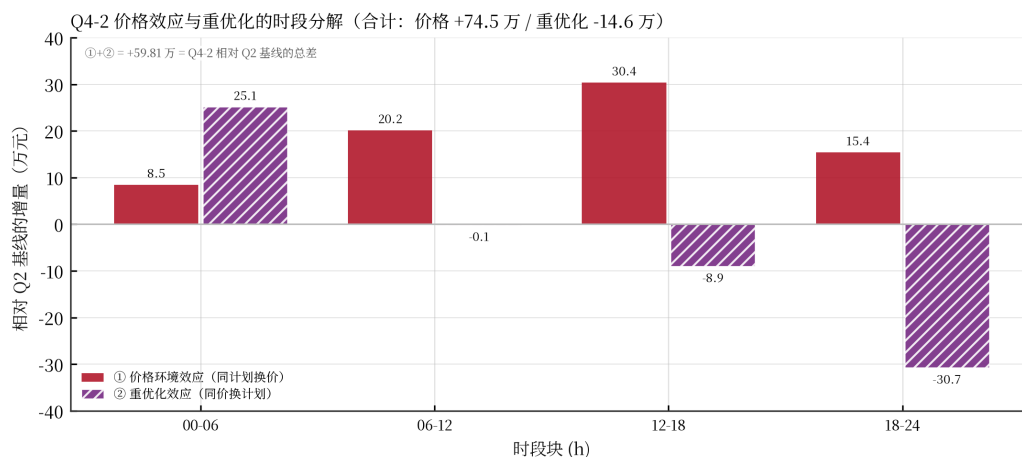


图 13 日前计划策略的分时段价格影响与重优化效果

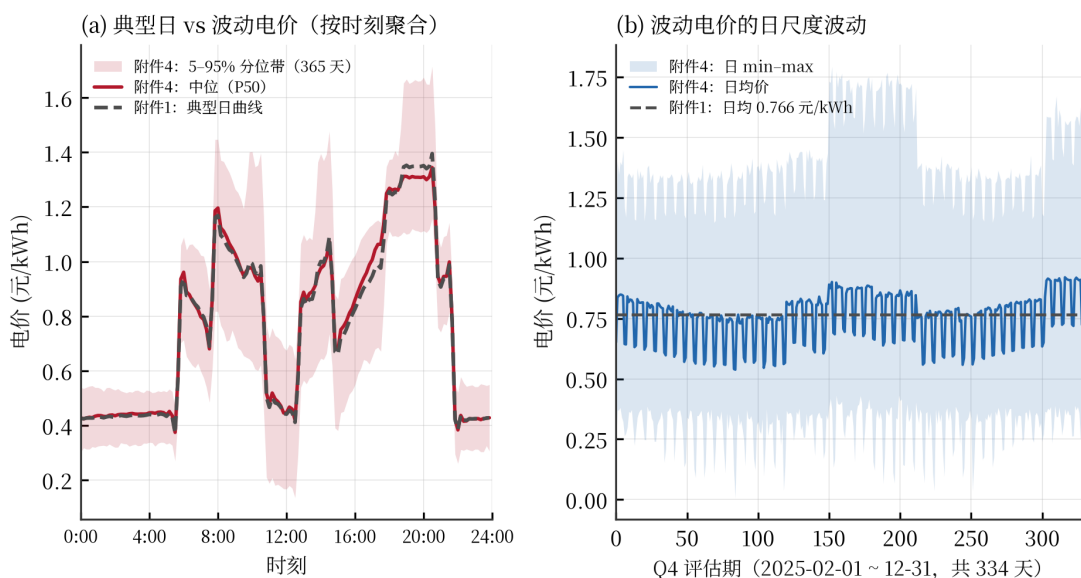


图 14 典型日电价与波动电价的时序特征

九、灵敏度与稳健性

9.1 灵敏度设计

第四至第七章的结论都建立在若干选择之上：终端库存怎么估值、储能效率取多少、预报质量如何、以及价格环境与策略各自贡献多少。为回答“这些选择会不会推翻结论”，本文按参数层 → 信息层 → 机制层三层做单轴扰动：每一条轴只改动一个因素，其余全部冻结。

两条统一的口径保证了这些比较是可比的：

1. 统一起点：所有扰动一律从 2025-02-01 0:00、储电量 6000 kWh 出发，评价期 334 天，不使用 1 月预热，与四问交付件同源；

2. 统一比较尺度：涉及终端库存的轴（终端估值、效率）会改变评价期末的储电量，若只看现金费用，会被”期末放空库存”污染。因此本文以库存修正成本为主判据：

$$J^{\text{corr}} = J_{\text{cash}} + \lambda(S_0 - S_T), \quad \lambda = 0.4720 \text{ 元/kWh} \quad (14)$$

现金费用只作辅助指标。这一条不是技术细节，而是结论正确性的前提，下一小节的例子说明：两种尺度会给出方向相反的判断。

9.2 参数层：终端估值与效率

（1）终端库存估值系数（问题二，重点）。把它在基准值 $\pm 20\%$ 范围内变动：

取值（元/kWh）	库存修正成本/元	相对基准	现金费用/元	期末储电量/kWh
0.3776	13,253,490.73	+0.016%	13,251,225.13	1,200
0.4720（本文）	13,251,420.69	—	13,252,341.09	7,950
0.5664	13,285,666.22	+0.258%	13,287,931.82	10,800

由此得到两条必须写清的结论：

- 结论对终端处理不敏感： $\pm 20\%$ 扰动下全年费用变化不超过 0.26%；
- 响应呈”平台一跳变”而非连续：估值偏低时期末库存被放空至 1,200 kWh，取本文值时为 7,950 kWh，偏高时顶到上界 10,800 kWh。更要紧的是：若只看现金费用，放空库存显得”更便宜”（13,251,225.13 元），但按库存修正成本它反而比本文取值贵 2,070.04 元，因为放空库存省下的是”把本来还会用的电提前用掉”。本文取 0.4720 位于稳健区间内，属于可辩护的建模选择（来源见 6.4），而非”最优取值”。

（2）储能单程效率（从简）。效率取 0.85 / 0.90 / 0.95 时，问题一至问题四的费用变化方向一致：降到 0.85 上升约 3.4%-4.2%，升到 0.95 下降约 3.4%-3.9%（问题一因要求当日循环最敏感）。需注意两点边界：问题三与问题四对应分支的效率轴只在 26 个代表日期上做状态冻结的局部筛查，属局部机制响应，不可当作全年稳健性；且幅度不可跨问直接比较。

9.3 信息层：预报质量如何传导（问题三）

把官方整点预报的误差强度整体缩放到 0.5 倍、1 倍（本文）、1.5 倍，得到：

误差强度 ρ	全年费用/元	相对本文	计划基底	调整相关	紧急购电量/kWh
0.5	12,805,045.44	-2.40%	12,360,747.72	275,369.75	36,680.69
1.0（本文）	13,120,194.06	—	12,381,239.55	542,052.30	42,672.22
1.5	13,478,837.67	+2.73%	12,424,724.30	818,388.11	51,024.15

结论有三层，关键在第二层：

1. 费用对预报质量有实质敏感度 ($\pm 2.4\%$ - 2.7%)；
2. 误差主要通过调整层与紧急购电传导，调整相关成本变化约 ± 27 万元、紧急购电费变化 $-2.8/+3.9$ 万元，而计划基底仅变化 -0.17% 至 $+0.35\%$ ；
3. 因此，“提高预报精度”的收益主要体现为减少临时调整与紧急购电，而不是改变日前计划本身。这正解释了为什么问题三的方案把重点放在滚动更新与执行规则上。

（该轴的扰动施加在官方预报的原生整点节点上，节点级平均绝对误差严格按 0.5:1:1.5 成比例，说明扰动确实作用在预报输入而非事后拟合。）

9.4 机制层：价格环境与策略重优化各占多少（问题四）

用两套计划 $\times$ 两套结算价格交叉结算（同一执行规则，与价格无关），把问题三口径的费用上升拆开：

效应	金额/元	含义
价格环境效应	+760,032.97	同一计划在新价格下结算的代价
策略重优化效应	-153,193.38	在新价格下重算计划的抵消量
合计	+606,839.60	即问题四对应分支相对问题三的上升

分解残差为 0（按未舍入值计算）；反向对照（旧价格下重优化 +104,956.32、新计划的价格环境 +501,883.28）同时说明价格环境效应依赖计划来源，属机制诊断，不能读作“改进”。

9.5 消融与结构对照

- 问题二的信息结构（五个对照臂）：完整阶梯表见附录 A。关键是同对象对照，把“均值点预报、不加保守裕度”（14,178,004.00 元）与本文方案相比，裕度价值 92.57 万元（6.53%）；而“4 情景平均”（13,396,023.55 元）属不同对象的方法对照，不能用来算裕度价值。
- 问题三的预报更新价值（四组时点）：仅 0:00 为 13,564,151.84 元，加入 6:00/12:00/18:00 后依次为 13,446,385.03 / 13,125,320.04 / 13,120,194.06 元，合计节省 443,957.78 元（3.27%），边际价值排序为 12:00 > 6:00 > 18:00；紧急购电量由 98,295.60 降至 42,672.22 kWh（-56.6%）。该结论直接回答了题面“是否需要引入其他时刻预报”的追问。
- 问题四的裕度结构（M0-M3）：把标量裕度换成日型条件、逐时段、分块三类结构，费用变化仅 -0.21% 至 $+0.05\%$ ；两侧问题口径的比值几乎不动（1.0451 $\rightarrow$ 1.0456）。结论：裕度结构不是问题二与问题四差异的来源。

9.6 结论的边界

- 1. 上述结论均在同问题、同口径、单轴条件下成立；跨问费用差只作并列与机制分解，不做减法，也不解释为某一种方法的优劣；
- 2. 涉及终端库存的两条轴（估值系数、效率）必须用库存修正成本判断，现金费用仅作参考；
- 3. 效率轴在问题三与问题四对应分支上为 26 日局部筛查，只能表述为局部机制响应；
- 4. 本文未做的探索（更细的参数网格、更多情景集、其他预报机制）在附录中列出，涉及样本内调参的方案已主动放弃（见 6.5）。

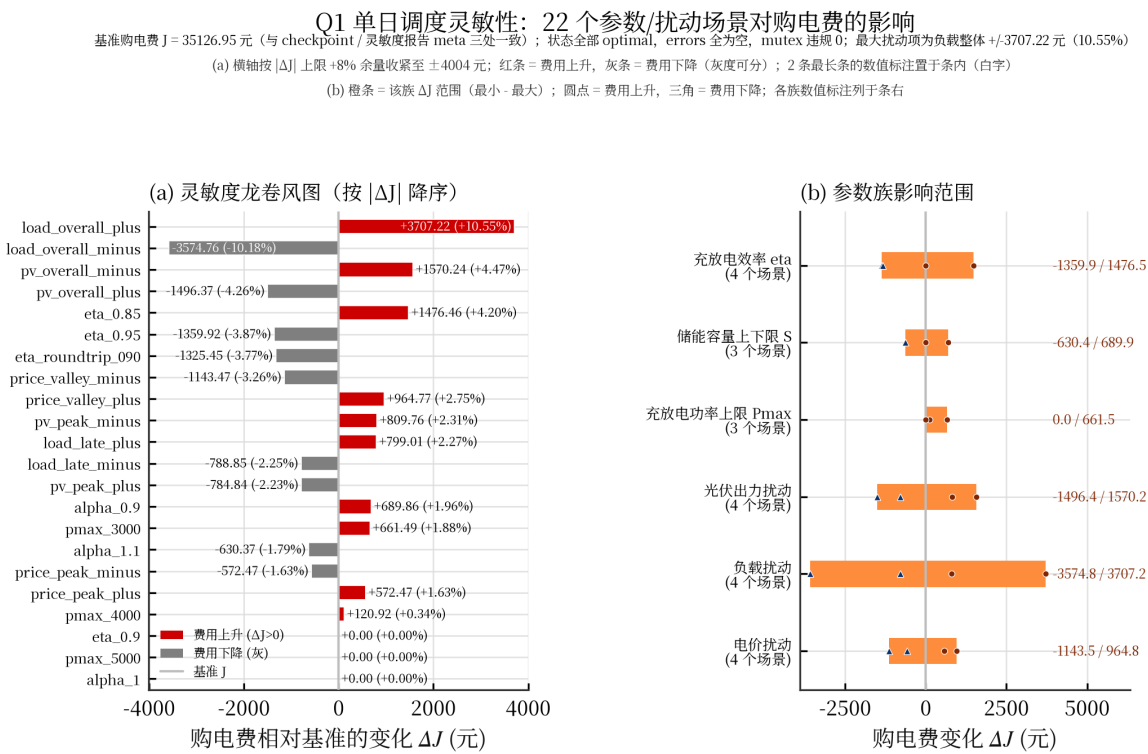


图 15 问题一不同参数扰动下的购电费用变化

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于程序调试与格式整理、文档与图表排版、按队员给定的判据编写重复性计算脚本，以及就队员已形成的建模方案征求第三方质询意见；问题的理解、建模思路与口径裁定、算法设计、结果分析与结论均由队员独立完成。详细使用情况见支撑材料《AI 工具使用详情.pdf》。

参考文献

[1] 姜启源, 谢金星, 叶俊. 数学模型 [M]. 5 版. 北京: 高等教育出版社, 2018.

- [2] 司守奎, 孙玺菁. 数学建模算法与应用 [M]. 3 版. 北京: 国防工业出版社, 2021.
- [3] 刘宝碁, 赵瑞清, 王纲. 不确定规划及应用 [M]. 北京: 清华大学出版社, 2003.
- [4] 王锡凡, 方万良, 杜正春. 现代电力系统分析 [M]. 北京: 科学出版社, 2003.
- [5] ARROW K J, HARRIS T, MARSHAK J. Optimal inventory policy[J]. *Econometrica*, 1951, 19(3): 250-272.
- [6] BELLMAN R. Dynamic Programming[M]. Princeton: Princeton University Press, 1957.
- [7] BEN-TAL A, GORYASHKO A, GUSLITZER E, et al. Adjustable robust solutions of uncertain linear programs[J]. *Mathematical Programming*, 2004, 99(2): 351-376.
- [8] BIRGE J R, LOUVEAUX F. Introduction to Stochastic Programming[M]. 2nd ed. New York: Springer, 2011.
- [9] CELADES E, PÉREZ E, APARICIO N, et al. Tool for optimization of sale and storage of energy in wind farms[J]. *Mathematics and Computers in Simulation*, 2024, 224: 2-18. DOI: 10.1016/j.matcom.2023.03.010.
- [10] CHEN C, DUAN S, CAI T, et al. Smart energy management system for optimal microgrid economic operation[J]. *IET Renewable Power Generation*, 2011, 5(3): 258-267. DOI: 10.1049/iet-rpg.2010.0052.
- [11] CONEJO A J, CARRIÓN M, MORALES J M. Decision Making Under Uncertainty in Electricity Markets[M]. New York: Springer, 2010.
- [12] DANESHVAR M, MOHAMMADI-IVATLOO B, ZARE K, et al. Two-stage robust stochastic model scheduling for transactive energy based renewable micro-grids[J]. *IEEE Transactions on Industrial Informatics*, 2020, 16(11): 6857-6867. DOI: 10.1109/TII.2020.2973740.
- [13] DANTZIG G B. Linear Programming and Extensions[M]. Princeton: Princeton University Press, 1963.
- [14] DUFO-LÓPEZ R. Optimisation of size and control of grid-connected storage under real time electricity pricing conditions[J]. *Applied Energy*, 2015, 140: 395-408. DOI: 10.1016/j.apenergy.2014.12.012.
- [15] FAZLALIPOUR P, EHSAN M, MOHAMMADI-IVATLOO B. Risk-aware stochastic bidding strategy of renewable micro-grids in day-ahead and real-time markets[J]. *Energy*, 2019, 171: 689-700. DOI: 10.1016/j.energy.2018.12.173.

- [16] GRIMALDI A, MINUTO F D, BROUWER J, et al. Profitability of energy arbitrage net profit for grid-scale battery energy storage considering dynamic efficiency and degradation using a linear, mixed-integer linear, and mixed-integer non-linear optimization approach[J]. *Journal of Energy Storage*, 2024, 95: 112380. DOI: 10.1016/j.est.2024.112380.
- [17] GULIN M, PAVLOVIĆ T, VAŠAK M. A one-day-ahead photovoltaic array power production prediction with combined static and dynamic on-line correction[J]. *Solar Energy*, 2017, 142: 49-60. DOI: 10.1016/j.solener.2016.12.008.
- [18] HOWARD R A. Information value theory[J]. *IEEE Transactions on Systems Science and Cybernetics*, 1966, 2(1): 22-26.
- [19] HYNDMAN R J, ATHANASOPOULOS G. *Forecasting: Principles and Practice*[M]. 2nd ed. Melbourne: OTexts, 2018.
- [20] KAUFMAN L, ROUSSEEUW P J. *Finding Groups in Data: An Introduction to Cluster Analysis*[M]. New York: Wiley, 1990.
- [21] KLEYWEGT A J, SHAPIRO A, HOMEM-DE-MELLO T. The sample average approximation method for stochastic discrete optimization[J]. *SIAM Journal on Optimization*, 2002, 12(2): 479-502.
- [22] LI Y, PENG J, JIA H, et al. Optimal battery schedule for grid-connected photovoltaic-battery systems of office buildings based on a dynamic programming algorithm[J]. *Journal of Energy Storage*, 2022, 50: 104557. DOI: 10.1016/j.est.2022.104557.
- [23] LUJANO-ROJAS J M, DUFO-LÓPEZ R, BERNAL-AGUSTÍN J L, et al. Optimizing daily operation of battery energy storage systems under real-time pricing schemes[J]. *IEEE Transactions on Smart Grid*, 2017, 8(1): 316-330. DOI: 10.1109/TSG.2016.2602268.
- [24] MALYSZ P, SIROUSPOUR S, EMADI A. MILP-based rolling horizon control for microgrids with battery storage[C]//*IECON 2013 - 39th Annual Conference of the IEEE Industrial Electronics Society*. Vienna: IEEE, 2013: 2099-2104.
- [25] NADARAYA E A. On estimating regression[J]. *Theory of Probability and Its Applications*, 1964, 9(1): 141-142.
- [26] PARISIO A, RIKOS E, GLIELMO L. A model predictive control approach to micro-grid operation optimization[J]. *IEEE Transactions on Control Systems Technology*, 2014, 22(5): 1813-1827. DOI: 10.1109/TCST.2013.2295737.

- [27] SIOSHANSI R, DENHOLMP, JENKIN T, et al. Estimating the value of electricity storage in PJM: arbitrage and some welfare effects[J]. Energy Economics, 2009, 31(2): 269-277. DOI: 10.1016/j.eneco.2008.10.005.
- [28] UMEZOR E C, TRIFKOVIC M. Operational scheduling of microgrids via parametric programming[J]. Applied Energy, 2016, 180: 672-681. DOI: 10.1016/j.apenergy.2016.08.009.
- [29] WATSON G S. Smooth regression analysis[J]. Sankhyā: The Indian Journal of Statistics, Series A, 1964, 26(4): 359-372.
- [30] WU J, XING X, LIU X, et al. Energy management strategy for grid-tied microgrids considering the energy storage efficiency[J]. IEEE Transactions on Industrial Electronics, 2018, 65(12): 9539-9549. DOI: 10.1109/TIE.2018.2818660.
- [31] XU Y, XIE L, SINGH C. Optimal scheduling and operation of load aggregator with electric energy storage in power markets[C]//2010 North American Power Symposium. Arlington, TX: IEEE, 2010: 1-7.
- [32] ZHANG X, HUANG C, SHEN J. Energy optimal management of microgrid with high photovoltaic penetration[J]. IEEE Transactions on Industry Applications, 2023, 59(1): 128-137. DOI: 10.1109/TIA.2022.3208885.
- [33] ZHANG Y, MENG F, WANG R, et al. A stochastic MPC based approach to integrated energy management in microgrids[J]. Sustainable Cities and Society, 2018, 41: 349-362. DOI: 10.1016/j.scs.2018.05.044.

附录 A 指定日期明细表

表 4 问题二指定日期的储能充放电量与日界储电量（按题面表 2 格式）

日期	时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
2025/3/20	0:00-4:00	2,332.84	0.00	4:00-8:00	1,010.17	5,526.08
	8:00-12:00	5,937.65	2,777.38	12:00-16:00	4,451.16	364.61
	16:00-20:00	41.73	5,790.89	20:00-24:00	7,500.00	2,773.02
	0:00 储电量	7,950.44	24:00 储电量	7,950.00		
2025/6/21	0:00-4:00	0.00	1,311.73	4:00-8:00	344.45	4,405.91
	8:00-12:00	9,849.77	0.00	12:00-16:00	0.00	0.00
	16:00-20:00	16.05	6,127.58	20:00-24:00	7,500.00	2,485.90
	0:00 储电量	7,978.14	24:00 储电量	7,993.91		
2025/9/23	0:00-4:00	2,333.33	0.00	4:00-8:00	889.56	5,635.92
	8:00-12:00	6,293.08	1,896.62	12:00-16:00	3,525.40	465.97
	16:00-20:00	0.93	5,629.73	20:00-24:00	7,500.00	3,011.02
	0:00 储电量	7,950.00	24:00 储电量	7,950.00		
2025/12/21	0:00-4:00	2,333.32	0.00	4:00-8:00	1,666.66	1,483.03
	8:00-12:00	5,855.25	7,112.37	12:00-16:00	6,663.91	2,223.29
	16:00-20:00	3.93	5,797.47	20:00-24:00	7,500.00	2,842.53
	0:00 储电量	7,950.01	24:00 储电量	7,950.00		

表 7 问题三指定日期的储能充放电与日界储电量（按题面表 2 格式，网格表）

日期	时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
2025/3/20	0:00-4:00	2,332.27	0.00	4:00-8:00	841.07	6,073.62
	8:00-12:00	4,478.39	2,468.26	12:00-16:00	6,083.54	248.21
	16:00-20:00	197.69	5,640.41	20:00-24:00	7,500.00	2,930.22
2025/6/21	0:00-4:00	0.00	1,333.46	4:00-8:00	399.38	4,389.23
	8:00-12:00	9,809.90	0.00	12:00-16:00	0.00	0.00
	16:00-20:00	24.66	6,090.47	20:00-24:00	7,500.00	2,485.90
2025/9/23	0:00-4:00	2,333.33	0.00	4:00-8:00	843.92	6,343.53
	8:00-12:00	6,128.07	1,895.25	12:00-16:00	4,491.03	371.26
	16:00-20:00	0.00	5,602.90	20:00-24:00	7,500.00	3,037.09
2025/12/21	0:00-4:00	2,330.27	0.00	4:00-8:00	1,664.10	1,506.25
	8:00-12:00	5,269.27	7,706.18	12:00-16:00	8,014.21	2,220.11
	16:00-20:00	0.00	5,797.47	20:00-24:00	7,500.00	2,842.53

表 9 问题四波动电价下的全年结果与归因（网格表）

指标	对应问题二分支	对应问题三分支
全年费用/元	13,850,454.64	13,727,033.66
对应固定电价基线/元	13,252,341.09	13,120,194.06
相对基线/元	+598,113.55	+606,839.60
价格环境效应/元	+744,522.48	+760,032.97
策略重优化效应/元	-146,408.94	-153,193.38

表 10 问题四指定日期的全天购电量与购电费（按题面表 1 格式，网格表）

日期	全天计划购电量（对应问题二分支）/kWh	全天购电费/元	全天计划购电量（对应问题三分支）/kWh	全天购电费/元
2025/3/20	67,948.54	67,948.54	71,794.22	71,794.22
2025/6/21	35,601.06	35,601.06	35,317.51	35,317.51
2025/9/23	69,050.22	69,050.22	71,846.20	71,846.20
2025/12/21	90,210.08	90,210.08	91,121.97	91,121.97

附录 B 核心源程序清单

本附录给出结果生成所依赖的核心可运行程序，运行环境为 Python 3.12 与 numpy、scipy。为便于排版，附录副本中的数学符号已按源码语义改写为 ASCII 写法，功能与支撑材料中的原件一致；全部源程序与数据随支撑材料提交。

Q1 问题一：144 时段线性规划求解

```
"""Q1 两阶段 LP 求解与 MILP 兜底.

建模口径严格对齐 ``Q1_1.0.md`` 3、10:

- 变量顺序 ``x = [G(0..143), C(0..143), D(0..143), R(0..143), S(0..144)]``.
- ``S_t`` 为时间节点 ``t*10min`` 处的储电量 (``S_0``=0:00, ``S_144``=24:00).
- 储能递推 ``S_{t+1} = S_t + eta_c*C_t - D_t/eta_d``.
"""

from __future__ import annotations

from typing import Dict, Tuple

import numpy as np
from scipy.optimize import Bounds, LinearConstraint, linprog, milp

from q1_common import (
    ETA_C,
    ETA_D,
    T,
    EBAR,
    SO,
    S_T,
    S_MIN,
    S_MAX,
)

# 变量块偏移
_OFF_G = 0
_OFF_C = T
_OFF_D = 2 * T
_OFF_R = 3 * T
_OFF_S = 4 * T
N_VAR = 4 * T + (T + 1)  # 721

_LP_STATUS = {0: "optimal", 1: "iteration_limit", 2: "infeasible", 3: "unbounded", 4: "numerical"}

def _lp_status_str(code: int) -> str:
    return _LP_STATUS.get(int(code), f"status_{code}")

def _idx_g(t: int) -> int:
    return _OFF_G + t

def _idx_c(t: int) -> int:
    return _OFF_C + t

def _idx_d(t: int) -> int:
    return _OFF_D + t

def _idx_r(t: int) -> int:
    return _OFF_R + t

def _idx_s(t: int) -> int:
    return _OFF_S + t
```

```

def build_model(
    price: np.ndarray,
    load_e: np.ndarray,
    pv_e: np.ndarray,
    eta_c: float = ETA_C,
    eta_d: float = ETA_D,
    s0: float = S0,
    s_t: float = S_T,
    s_min: float = S_MIN,
    s_max: float = S_MAX,
    ebar: float = EBAR,
) -> Tuple[np.ndarray, np.ndarray, np.ndarray, list]:
    """构造 Q1 线性规划的等式矩阵、右端项、目标向量与变量边界。

    Returns
    -----
    (A_eq, b_eq, c1, bounds)
        ``c1`` 为第一阶段成本目标系数；第二阶段目标 ``c2`` 由调用方自行构造。
    """
    price = np.asarray(price, dtype=float)
    load_e = np.asarray(load_e, dtype=float)
    pv_e = np.asarray(pv_e, dtype=float)
    if price.shape != (T,) or load_e.shape != (T,) or pv_e.shape != (T,):
        raise ValueError("price/load_e/pv_e 长度必须为 T=144")

    n_eq = 2 * T + 2
    A_eq = np.zeros((n_eq, N_VAR), dtype=float)
    b_eq = np.zeros(n_eq, dtype=float)

    # 1) 能量平衡  $G_t + D_t - C_t - R_t = Load_e - PV_e$ 
    for t in range(T):
        A_eq[t, _idx_g(t)] = 1.0
        A_eq[t, _idx_d(t)] = 1.0
        A_eq[t, _idx_c(t)] = -1.0
        A_eq[t, _idx_r(t)] = -1.0
        b_eq[t] = load_e[t] - pv_e[t]

    # 2) 储能递推  $S_{t+1} - S_t - eta_c * C_t + D_t / eta_d = 0$ 
    for t in range(T):
        row = T + t
        A_eq[row, _idx_s(t + 1)] = 1.0
        A_eq[row, _idx_s(t)] = -1.0
        A_eq[row, _idx_c(t)] = -eta_c
        A_eq[row, _idx_d(t)] = 1.0 / eta_d
        b_eq[row] = 0.0

    # 3)  $S_0 = s0$ 
    A_eq[2 * T, _idx_s(0)] = 1.0
    b_eq[2 * T] = s0

    # 4)  $S_T = s_t$ 
    A_eq[2 * T + 1, _idx_s(T)] = 1.0
    b_eq[2 * T + 1] = s_t

    # 第一阶段成本目标
    c1 = np.zeros(N_VAR, dtype=float)
    c1[_OFF_G:_OFF_G + T] = price

    # 变量边界
    lb = np.zeros(N_VAR, dtype=float)
    ub = np.full(N_VAR, np.inf, dtype=float)
    for t in range(T):
        ub[_idx_c(t)] = ebar
        ub[_idx_d(t)] = ebar
        ub[_idx_r(t)] = pv_e[t]
    lb[_OFF_S:_OFF_S + T + 1] = s_min
    ub[_OFF_S:_OFF_S + T + 1] = s_max
    bounds = list(zip(lb.tolist(), ub.tolist()))

    return A_eq, b_eq, c1, bounds

def _cost_cvec(price: np.ndarray) -> np.ndarray:
    """构造费用约束行的系数向量（作用于 G 块）。"""

```

```

v = np.zeros(N_VAR, dtype=float)
v[_OFF_G:_OFF_G + T] = np.asarray(price, dtype=float)
return v

def _throughput_cvec() -> np.ndarray:
    """第二阶段吞吐目标系数向量 (C + D) ."""
    v = np.zeros(N_VAR, dtype=float)
    v[_OFF_C:_OFF_C + T] = 1.0
    v[_OFF_D:_OFF_D + T] = 1.0
    return v

def _unpack(x: np.ndarray) -> Dict[str, np.ndarray]:
    return {
        "G": np.asarray(x[_OFF_G:_OFF_G + T], dtype=float),
        "C": np.asarray(x[_OFF_C:_OFF_C + T], dtype=float),
        "D": np.asarray(x[_OFF_D:_OFF_D + T], dtype=float),
        "R": np.asarray(x[_OFF_R:_OFF_R + T], dtype=float),
        "S": np.asarray(x[_OFF_S:_OFF_S + T + 1], dtype=float),
    }

def _linprog_solver_options() -> Dict:
    return {
        "presolve": True,
        "primal_feasibility_tolerance": 1e-9,
        "dual_feasibility_tolerance": 1e-9,
    }

def solve_two_stage_lp(
    price: np.ndarray,
    load_e: np.ndarray,
    pv_e: np.ndarray,
    eps_j: float,
    eta_c: float = ETA_C,
    eta_d: float = ETA_D,
    s0: float = S0,
    s_t: float = S_T,
    s_min: float = S_MIN,
    s_max: float = S_MAX,
    ebar: float = EBAR,
) -> Dict:
    """两阶段 LP 求解.

    Returns
    -----
    Dict
        包含 ``phase1`` / ``phase2`` 子字典、``x``、``J1_star``、``J_actual``、
        ``H`` 与解分量.若任一阶段非 optimal,则 ``status`` 非 ``optimal``.
    """
    price = np.asarray(price, dtype=float)
    load_e = np.asarray(load_e, dtype=float)
    pv_e = np.asarray(pv_e, dtype=float)
    A_eq, b_eq, c1, bounds = build_model(
        price, load_e, pv_e, eta_c, eta_d, s0, s_t, s_min, s_max, ebar
    )

    res1 = linprog(
        c1,
        A_eq=A_eq,
        b_eq=b_eq,
        bounds=bounds,
        method="highs",
        options=_linprog_solver_options(),
    )

    phase1 = {
        "status": _lp_status_str(res1.status),
        "status_code": int(res1.status),
        "success": bool(res1.success),
        "message": res1.message,
        "fun": None if res1.fun is None else float(res1.fun),
        "nit": int(res1.nit) if res1.nit is not None else None,
    }

```

```

if not res1.success or res1.x is None:
    return {
        "status": "not_optimal",
        "phase1": phase1,
        "phase2": None,
        "x": None,
        "J1_star": None,
        "J_actual": None,
        "H": None,
        "components": None,
        "method": "lp",
    }

j1_star = float(res1.fun)
c2 = _throughput_cvec()
A_ub = _cost_cvec(price).reshape(1, -1)
b_ub = np.array([j1_star + eps_j])

res2 = linprog(
    c2,
    A_ub=A_ub,
    b_ub=b_ub,
    A_eq=A_eq,
    b_eq=b_eq,
    bounds=bounds,
    method="highs",
    options=_linprog_solver_options(),
)

phase2 = {
    "status": _lp_status_str(res2.status),
    "status_code": int(res2.status),
    "success": bool(res2.success),
    "message": res2.message,
    "fun": None if res2.fun is None else float(res2.fun),
    "nit": int(res2.nit) if res2.nit is not None else None,
}

if not res2.success or res2.x is None:
    return {
        "status": "not_optimal",
        "phase1": phase1,
        "phase2": phase2,
        "x": None,
        "J1_star": j1_star,
        "J_actual": None,
        "H": None,
        "components": None,
        "method": "lp",
    }

comp = _unpack(res2.x)
j_actual = float(np.dot(price, comp["G"]))
_lock = j1_star + eps_j
if j_actual > _lock and (j_actual - _lock) <= 1e-9 * max(1.0, abs(j1_star)):
    # 二阶段费用约束在求解器可行容差内饱和, np.dot 重算产生 -1ULP 上浮, 钳到费用锁
    j_actual = _lock
h = float(np.sum(comp["C"] + comp["D"]))
return {
    "status": "optimal",
    "phase1": phase1,
    "phase2": phase2,
    "x": np.asarray(res2.x, dtype=float),
    "J1_star": j1_star,
    "J_actual": j_actual,
    "H": h,
    "components": comp,
    "method": "lp",
}

def solve_two_stage_milp(
    price: np.ndarray,
    load_e: np.ndarray,
    pv_e: np.ndarray,

```

```

eps_j: float,
eta_c: float = ETA_C,
eta_d: float = ETA_D,
s0: float = S0,
s_t: float = S_T,
s_min: float = S_MIN,
s_max: float = S_MAX,
ebar: float = EBAR,
) -> Dict:
    """两阶段 MILP 兜底 (HiGHS 内核) .

    在 LP 基础上引入二元变量 ``z_t`` 与互斥约束:
    ``C_t <= Ebar*z_t``、``D_t <= Ebar*(1-z_t)``.
    """
    price = np.asarray(price, dtype=float)
    load_e = np.asarray(load_e, dtype=float)
    pv_e = np.asarray(pv_e, dtype=float)

    # 变量顺序: [G, C, D, R, S, z], z 共 T 个
    z_off = N_VAR
    n_var = N_VAR + T
    c1 = np.zeros(n_var, dtype=float)
    c1[_OFF_G:_OFF_G + T] = price
    c2 = np.zeros(n_var, dtype=float)
    c2[_OFF_C:_OFF_C + T] = 1.0
    c2[_OFF_D:_OFF_D + T] = 1.0

    integrality = np.zeros(n_var, dtype=np.uint8)
    integrality[z_off:z_off + T] = 1

    lb = np.zeros(n_var, dtype=float)
    ub = np.full(n_var, np.inf, dtype=float)
    for t in range(T):
        ub[_idx_c(t)] = ebar
        ub[_idx_d(t)] = ebar
        ub[_idx_r(t)] = pv_e[t]
        ub[z_off + t] = 1.0
    lb[_OFF_S:_OFF_S + T + 1] = s_min
    ub[_OFF_S:_OFF_S + T + 1] = s_max

    # 等式约束 (同 LP)
    n_eq = 2 * T + 2
    A_eq = np.zeros((n_eq, n_var), dtype=float)
    b_eq = np.zeros(n_eq, dtype=float)
    for t in range(T):
        A_eq[t, _idx_g(t)] = 1.0
        A_eq[t, _idx_d(t)] = 1.0
        A_eq[t, _idx_c(t)] = -1.0
        A_eq[t, _idx_r(t)] = -1.0
        b_eq[t] = load_e[t] - pv_e[t]
    for t in range(T):
        row = T + t
        A_eq[row, _idx_s(t + 1)] = 1.0
        A_eq[row, _idx_s(t)] = -1.0
        A_eq[row, _idx_c(t)] = -eta_c
        A_eq[row, _idx_d(t)] = 1.0 / eta_d
        b_eq[row] = 0.0
    A_eq[2 * T, _idx_s(0)] = 1.0
    b_eq[2 * T] = s0
    A_eq[2 * T + 1, _idx_s(T)] = 1.0
    b_eq[2 * T + 1] = s_t

    # 不等式约束: 互斥 + 费用锁定 (第二阶段)
    n_mutex = 2 * T
    A_mutex = np.zeros((n_mutex, n_var), dtype=float)
    lb_mutex = np.full(n_mutex, -np.inf, dtype=float)
    ub_mutex = np.zeros(n_mutex, dtype=float)
    for t in range(T):
        # C_t - Ebar*z_t <= 0
        A_mutex[t, _idx_c(t)] = 1.0
        A_mutex[t, z_off + t] = -ebar
        # D_t + Ebar*z_t <= Ebar => D_t + Ebar*z_t - Ebar <= 0
        A_mutex[T + t, _idx_d(t)] = 1.0
        A_mutex[T + t, z_off + t] = ebar
        ub_mutex[T + t] = ebar

```

```

cost_row = np.zeros((1, n_var), dtype=float)
cost_row[0, _OFF_G:_OFF_G + T] = price

def _run(c, cost_ub):
    # 合并等式 (能量平衡 / SOC 递推 / 初末 SOC) 与不等式 (互斥 + 费用锁定)
    A_all = np.vstack([A_eq, A_mutex, cost_row])
    lb_all = np.concatenate([b_eq, lb_mutex, np.array([-np.inf])])
    ub_all = np.concatenate([b_eq, ub_mutex, np.array([cost_ub])])
    constraints = LinearConstraint(A_all, lb_all, ub_all)

    return milp(
        c,
        integrality=integrality,
        bounds=Bounds(lb, ub),
        constraints=constraints,
        options={"mip_rel_gap": 0.0, "presolve": True},
    )

# 第一阶段: 无费用锁定, 费用行上界取 +inf
res1 = _run(c1, np.inf)
status1 = int(res1.status)
if not res1.success or res1.x is None:
    return {
        "status": "not_optimal",
        "phase1_status": status1,
        "phase1_message": res1.message,
        "J1_star": None,
        "J_actual": None,
        "method": "milp",
    }

j1_star = float(res1.fun)
res2 = _run(c2, j1_star + eps_j)
status2 = int(res2.status)
if not res2.success or res2.x is None:
    return {
        "status": "not_optimal",
        "phase1_status": status1,
        "phase2_status": status2,
        "phase2_message": res2.message,
        "J1_star": j1_star,
        "J_actual": None,
        "method": "milp",
    }

comp = _unpack(res2.x[:N_VAR])
j_actual = float(np.dot(price, comp["G"]))
_lock = j1_star + eps_j
if j_actual > _lock and (j_actual - _lock) <= 1e-9 * max(1.0, abs(j1_star)):
    j_actual = _lock
h = float(np.sum(comp["C"] + comp["D"]))
return {
    "status": "optimal",
    "phase1_status": status1,
    "phase2_status": status2,
    "phase1_message": res1.message,
    "phase2_message": res2.message,
    "mip_gap": float(res2.mip_gap) if res2.mip_gap is not None else None,
    "J1_star": j1_star,
    "J_actual": j_actual,
    "H": h,
    "components": comp,
    "method": "milp",
}

```

Q2 问题二：计划层 MILP 求解

```

# -*- coding: utf-8 -*-
"""计划层求解器（独立于 q2_lp.py）. 三种口径：
lp : 线性 recourse (允许 e 与 C 同段)
milp : 全量二元互斥 e_t <= M z_t, C_t <= M(1-z_t) (每天 1008 个二元变量, 慢)
lazy : 惰性割先在无制 LP 上解, 找出真正 e>0 且 C>0 的 (k,t), 只为这些位置引入二元互斥, 重解:
        反复直到解不再违反互斥. 此时该解对“全量互斥 MILP”是**精确最优** (LP 松弛 <= MILP <= 该可行解) .
返回 (G, obj, info) .
"""

```

```

import sys
sys.path.append("D:\\CMUCU\\rag\\.deps")
import numpy as np
from scipy.sparse import csr_matrix
from scipy.optimize import linprog, milp, LinearConstraint, Bounds

T = 144; DT = 1/6; EBAR = 5000*DT; SMIN, SMAX, ETA = 1200.0, 10800.0, 0.9
TOL = 1e-7

def _build(Le, Pe, price, s0, lam, cut_pairs):
    """cut_pairs: list of (k, t) 需要施加互斥的位置 (每个占 1 个二元变量) ."""
    K, Tt = Le.shape
    nz = len(cut_pairs)
    nk = 5*Tt + 1 # C,D,R_PV,e = 4T ; S = T+1
    n = Tt + K*nk + nz
    c = np.zeros(n); c[0:Tt] = price
    re_, ce_, ve_, be = [], [], [], []
    ru, cu, vu, bu = [], [], [], []
    ri = ai = 0
    Me = float(max(1400.0, np.max(Le)*1.05)); Mc = float(EBAR)
    for k in range(K):
        off = Tt + k*nk
        oC, oD, oP, oE, oS = off, off+Tt, off+2*Tt, off+3*Tt, off+4*Tt
        for t in range(Tt):
            re_ += [ri]*5; ce_ += [t, oC+t, oD+t, oP+t, oE+t]; ve_ += [1.0, -1.0, 1.0, -1.0, 1.0]
            be.append(Le[k, t] - Pe[k, t]); ri += 1
        for t in range(Tt):
            re_ += [ri]*4; ce_ += [oS+t+1, oS+t, oC+t, oD+t]; ve_ += [1.0, -1.0, -ETA, 1/ETA]
            be.append(0.0); ri += 1
        re_ += [ri]; ce_ += [oS]; ve_ += [1.0]; be.append(s0); ri += 1
        c[oE:oE+Tt] = 5*price/K
        c[oS+Tt] = -lam/K
    for j, (k, t) in enumerate(cut_pairs):
        off = Tt + k*nk
        oC, oE = off, off+3*Tt
        zc = Tt + K*nk + j
        Me_kt = float(max(1.0, Le[k, t])) # 逐时段紧 big-M: e 不可能超过该时段负载电量
        ru += [ai, ai]; cu += [oE+t, zc]; vu += [1.0, -Me_kt]; bu.append(0.0); ai += 1
        ru += [ai, ai]; cu += [oC+t, zc]; vu += [1.0, Mc]; bu.append(Mc); ai += 1
    A_eq = csr_matrix((np.asarray(ve_), (np.asarray(re_), np.asarray(ce_))), shape=(ri, n))
    A_ub = csr_matrix((np.asarray(vu), (np.asarray(ru), np.asarray(cu))), shape=(ai, n)) if ai else None
    bd = [(0.0, None)]*Tt; integ = np.zeros(n)
    for k in range(K):
        off = Tt + k*nk
        bd += [(0.0, EBAR)]*Tt + [(0.0, EBAR)]*Tt
        + [(0.0, float(Pe[k, t])) for t in range(Tt)]
        + [(0.0, None)]*Tt + [(SMIN, SMAX)]*(Tt+1)
    bd += [(0.0, 1.0)]*nz
    if nz: integ[Tt + K*nk:] = 1
    return c, A_eq, np.asarray(be), A_ub, np.asarray(bu), bd, integ, nk, nz

def _solve_once(Le, Pe, price, s0, lam, cut_pairs, time_limit=180):
    c, Aeq, beq, Aub, bub, bd, integ, nk, nz = _build(Le, Pe, price, s0, lam, cut_pairs)
    if nz == 0:
        r = linprog(c, Aeq=Aeq, b_eq=beq, A_ub=Aub, b_ub=bub, bounds=bd, method="highs")
    else:
        cons = [LinearConstraint(Aeq, beq, beq)]
        if Aub is not None: cons.append(LinearConstraint(Aub, -np.inf, bub))
        r = milp(c, constraints=cons, integrality=integ,
            bounds=Bounds([-np.inf if b[0] is None else b[0] for b in bd],
                [np.inf if b[1] is None else b[1] for b in bd]),
            options={"time_limit": time_limit, "mip_rel_gap": 1e-7})
    if not r.success and r.x is None:
        return None, None, nk, dict(status=int(r.status), msg=str(r.message))
    return r.x, float(r.fun), nk, dict(status=int(r.status), msg=str(r.message)[:80],
        success=bool(r.success))

def _extract(x, K, Tt, nk):
    G = x[0:Tt]; Es = np.zeros(Tt); Cs = np.zeros(Tt)
    both = []
    for k in range(K):
        off = Tt + k*nk
        e = x[off+3*Tt:off+4*Tt]; C = x[off:off+Tt]

```

```

        Es += e/K; Cs += C/K
        idx = np.where((e > TOL) & (C > TOL))[0]
        both += [(k, int(t)) for t in idx]
    return G, Es, Cs, both

def solve(Le, Pe, price, s0, lam=0.4720, mode="lazy", max_iter=40):
    K, Tt = Le.shape
    if mode == "milp":
        cuts = [(k, t) for k in range(K) for t in range(Tt)]
        x, obj, nk, info = _solve_once(Le, Pe, price, s0, lam, cuts, time_limit=300)
        if x is None: return None, None, dict(mode="milp", fail=True, **info)
        G, Es, Cs, both = _extract(x, K, Tt, nk)
        return G, obj, dict(mode="milp", cuts=len(cuts), n_viol=len(both), **info)
    cuts = []
    for it in range(max_iter):
        x, obj, nk, info = _solve_once(Le, Pe, price, s0, lam, cuts)
        if x is None:
            return None, None, dict(iters=it, cuts=len(cuts), fail=True, **info)
        G, Es, Cs, both = _extract(x, K, Tt, nk)
        if mode == "lp":
            return G, obj, dict(iters=1, cuts=0, n_viol=len(both), status=0)
        if not both:
            return G, obj, dict(iters=it+1, cuts=len(cuts), n_viol=0, status=0)
        # 只保留尚未加入的割
        new = [p for p in both if p not in set(cuts)]
        cuts += new
    return None, None, dict(iters=max_iter, cuts=len(cuts), not_converged=True)

```

Q3 问题三：物化驱动（4 小时段）

```

# -*- coding: utf-8 -*-
"""B2 物化驱动：逐段 JSONL -> 解法 JSON -> ``result3.xlsx``（模板另存副本）。

本脚本只做「物化 + 前后哈希取证」, **不做求解、不做口径裁定**, 也不修改模板。

冻结口径（照抄 7.5 ``q3_materialize.py``, 不得自创）
-----
- 列映射 = 位置映射 ``col = 2 + t``（模板表头整体错位一格, 禁止按标签对齐）:
  第 145 列 = 第 144 段, 第 146 列 = 全天购电量, 第 147 列 = 全天购电费
  (**方案 A, 2026-09-13 统一**: 第 147 列 = 本表购电量 * 该时段电价, 第 146/147 列均保留 4 位小数;
  全年总购电费用（计划基础 + 调整相关 + 紧急）不落在表内, 见交付说明与论文）。
- 报账读法 ``C``: ``p*min(G0,A) + 0.5p(G0-A)+ + 1.5p(A-G0)+ + 5p*H``。
- 表 1 = ``G0``、表 2 = ``A``、表 3 = 四小时块充放电量、表 4 = 紧急购电真实发生段
  （同日相邻合并、跨日不合并、日期只在当日首段写一次）。

实现方式
-----
物化引擎直接 **以库方式只读引用** 7.5 的 ``q3_materialize.py``（唯一口径来源），
调用前先复算其 SHA-256 与冻结值比对, 不一致即中止（防止 7.5 被改动后口径漂移）。
本仓库的格式转换（JSONL -> 顶层含 ``days`` 数组）由同目录 ``q76_jsonl_to_days.py`` 完成。

用法
----

python q76_materialize.py --seg <seg.jsonl> --solution <solution.json> \\\
    --out <result3.xlsx> --template <附件5/result3.xlsx> \\\
    --price <q1_clean.csv> --reading C --report <报告.json>

退出码: 0 成功; 2 输入问题; 3 写前自检不通过 / 引擎哈希不符。
"""
from __future__ import annotations

import argparse
import datetime as _dt
import hashlib
import json
import sys
from pathlib import Path

CODE_DIR = Path(__file__).resolve().parent
Q75_CODE = Path(r"D:\CMUCU\7.5对话\code")
# 冻结哈希（2026-09-12 复算, 与 _sub/q76_pioneer_B.md 记录一致）
FROZEN_ENGINE_SHA256 = "4B7CAA678F15444579206DEA85BD72357A6258D3390B9083954F4E86891501FF"

```

```

DEFAULT_TEMPLATE = r"D:\CMUCU\赛题\C题\附件\附件5\result3.xlsx"
DEFAULT_PRICE_CSV = r"D:\CMUCU\B对话\clean\q1_clean.csv"

def sha256_of(path) -> str:
    """文件 SHA-256 (大写十六进制) ."""
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b''):
            h.update(chunk)
    return h.hexdigest().upper()

def load_engine():
    """只读引入 7.5 冻结物化引擎,并复算哈希.返回 ``engine_module, sha256``."""
    target = Q75_CODE / "q3_materialize.py"
    if not target.exists():
        raise FileNotFoundError(f"冻结物化引擎不存在: {target}")
    sys.path.insert(0, str(Q75_CODE))
    import q3_materialize as engine # noqa: E402 (只读引用,不改 7.5)
    # * 2026-09-12 整改: 7.5 的 engine 把时段块写成 BLOCK=36 (6 小时),
    # 而模板表 2 的「指定时间段」是 4 小时块 (0:00-4:00 / 4:00-8:00 / ...).
    # 结果会导致第 5、6 行 (16:00-20:00、20:00-24:00) 恒为 0、并整体错位.
    # 在本侧覆盖为 24 (不改 7.5 任何文件),并断言 24*6==144.
    if hasattr(engine, "BLOCK"):
        engine.BLOCK = 24
        assert 24 * 6 == 144, "block override sanity"
        print("[q76] override engine.BLOCK = 24 (4h blocks, 6 per day)")

    got = sha256_of(engine.__file__)
    if got != FROZEN_ENGINE_SHA256:
        raise ValueError(
            "物化引擎 SHA-256 与冻结值不一致,拒绝据此物化: \n"
            f" 实际 {got}\n 冻结 {FROZEN_ENGINE_SHA256}")
    return engine, got

def run(seg, solution, out, template=DEFAULT_TEMPLATE, price_csv=DEFAULT_PRICE_CSV,
        reading="C", force=False) -> dict:
    """转换 (可选) -> 物化 -> 哈希取证.返回物化报告 dict."""
    tpl = Path(template)
    if not tpl.exists():
        raise FileNotFoundError(f"模板不存在: {tpl}")
    if Path(out).resolve() == tpl.resolve():
        raise ValueError("拒绝覆盖模板: --out 不能等于模板路径")

    engine, engine_sha = load_engine()
    tpl_sha_before = sha256_of(tpl)
    tpl_bytes_before = tpl.stat().st_size

    conv = None
    if seg is not None:
        sys.path.insert(0, str(CODE_DIR))
        import q76_jsonl_to_days as CV # noqa: E402

        conv = CV.convert(seg, solution, price_csv=price_csv)

    rep = engine.materialize(str(solution), str(out), template=str(tpl),
                            price_csv=str(price_csv), reading=reading, force=force)

    tpl_sha_after = sha256_of(tpl)
    tpl_bytes_after = tpl.stat().st_size
    out_p = Path(out)
    return {
        "generated_at": _dt.datetime.now().isoformat(timespec="seconds"),
        "pipeline": {
            "seg_jsonl": str(seg) if seg else None,
            "solution_json": str(solution),
            "out_xlsx": str(out),
            "template": str(tpl),
            "price_csv": str(price_csv),
            "reading": str(reading).upper(),
        },
        "conversion": conv,
        "materialize": rep,
        "engine": {"path": str(engine.__file__), "sha256": engine_sha,

```

```

        "frozen_sha256": FROZEN_ENGINE_SHA256,
        "sha_match": engine_sha == FROZEN_ENGINE_SHA256},
    "template_integrity": {
        "sha256_before": tpl_sha_before,
        "sha256_after": tpl_sha_after,
        "bytes_before": tpl_bytes_before,
        "bytes_after": tpl_bytes_after,
        "unchanged": (tpl_sha_before == tpl_sha_after
                       and tpl_bytes_before == tpl_bytes_after),
    },
    "out_xlsx": {
        "path": str(out_p),
        "exists": out_p.exists(),
        "bytes": out_p.stat().st_size if out_p.exists() else None,
        "sha256": sha256_of(out_p) if out_p.exists() else None,
    },
}

def main(argv=None) -> int:
    ap = argparse.ArgumentParser(
        description="B2 物化: 逐段 JSONL -> 解法 JSON -> result3.xlsx (另存副本)")
    ap.add_argument("--seg", default=None, help="主线 --segments-out 落的 JSONL (给了就先转换)")
    ap.add_argument("--solution", required=True, help="解法 JSON (顶层含 days: --seg 给定时为转换输出)")
    ap.add_argument("--out", required=True, help="输出的 xlsx (不得等于模板)")
    ap.add_argument("--template", default=DEFAULT_TEMPLATE, help="附件5 模板 (只读)")
    ap.add_argument("--price", default=DEFAULT_PRICE_CSV, help="附件1 清洗件 (144 段电价)")
    ap.add_argument("--reading", default="C", choices=["A", "C", "a", "c"], help="报账读法 (默认 C)")
    ap.add_argument("--report", default=None, help="物化报告 JSON")
    ap.add_argument("--force", action="store_true", help="允许覆盖已存在的输出")
    args = ap.parse_args(argv)

    try:
        rep = run(args.seg, args.solution, args.out, template=args.template,
                  price_csv=args.price, reading=args.reading, force=args.force)
    except FileNotFoundError as e:
        print(f"[ERROR] 输入缺失: {e}", file=sys.stderr)
        return 2
    except FileExistsError as e:
        print(f"[ERROR] {e}", file=sys.stderr)
        return 2
    except (ValueError, KeyError, TypeError, json.JSONDecodeError) as e:
        print(f"[ERROR] 数据/口径不合法: {e}", file=sys.stderr)
        return 3

    m = rep["materialize"]
    print("[OK] B2 物化完成")
    print(f" 输出      : {m['out']}")
    print(f" 区间      : {m['date_first']} - {m['date_last']} ({m['n_days']} 天)")
    print(f" 列映射     : {m['col_map']['col_of_t']} (首段列 {m['col_map']['first_t_col']})")
    print(f" 充放电行数: {m['sheets']['soc']['rows']}")
    print(f" 紧急购电行: {m['sheets']['emergency']['rows']}")
    f = (合并前 {m['sheets']['emergency']['segments_merged']} 段)
    print(f" 模板未改   : {rep['template_integrity']['unchanged']} "
          f"({rep['template_integrity']['sha256_after'][:16]})")
    print(f" 引擎哈希   : {rep['engine']['sha_match']} ({rep['engine']['sha256'][:16]})")
    if args.report:
        Path(args.report).parent.mkdir(parents=True, exist_ok=True)
        Path(args.report).write_text(json.dumps(rep, ensure_ascii=False, indent=1),
                                     encoding="utf-8")
        print(f" 物化报告   : {args.report}")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

Q4 问题四：交叉结算与归因分解

```

# -*- coding: utf-8 -*-

"""2x2 反事实交叉结算 (GPT 实验建议: "最能封口"的一张表)
行 = 计划来源 (附件1 价下优化 / 附件4 价下优化): 列 = 结算价格 (附件1 / 附件4)
执行器 E1 只依赖 (G, 实际 L/PV), 与价格无关 H 由 G 决定, 价格只影响计费.
只读: 只写 8 对话\output.

```

```

"""
import sys, os, json
import numpy as np
import pandas as pd
from scipy.optimize import linprog
import importlib.util

sys.path.append("D:\\CMUCU\\rag\\.deps")
sys.path.append(r"D:\CMUCU\5对话\code")

def load(n, p):
    s = importlib.util.spec_from_file_location(n, p)
    m = importlib.util.module_from_spec(s)
    s.loader.exec_module(m)
    return m

mm = load("mm", r"D:\CMUCU\5对话\code\q2_v2_micro.py")
mp = load("mp", r"D:\CMUCU\5对话\code\q2_milp_plan.py")
L, P, PRICE1 = mm.load_all()
T, DT, SO = mm.T, mm.DT, mm.SO
EBAR, SMIN, SMAX, ETA = mm.EBAR, mm.SMIN, mm.SMAX, mm.ETA
LAM = 0.4720
K, WIN, CAP = 4, 28, 0.35
DAYS = list(range(31, 365))
OUT = r"D:\CMUCU\8对话\output"
P4 = pd.read_csv(r"D:\CMUCU\B对话\clean\attachment4_clean.csv").iloc[:, 1:].to_numpy(float)

def margin(d, relq=0.20):
    rels = []
    for tau in range(max(K, d - WIN), d):
        kk = list(range(max(0, tau - K), tau))
        if len(kk) == K:
            f = (P[kk] * DT).mean(axis=0)
            m = f > 1.0
            if m.any():
                rels.append((P[tau] * DT - f)[m] / f[m])
    if not rels:
        return 0.0
    return float(np.clip(-np.quantile(np.concatenate(rels), relq), 0.0, CAP))

def build_plan(price_of_day, tag):
    """按给定价格优化逐日计划,并对实际数据执行 E1,返回逐日 G/H 与各自价格下的现金."""
    s = SO
    Gs = np.zeros((len(DAYS), T))
    Hs = np.zeros((len(DAYS), T))
    for i, d in enumerate(DAYS):
        ks = list(range(max(0, d - K), d))
        base = (P[ks] * DT).mean(axis=0)
        Pe_d = np.clip(base * (1 - margin(d)), 0.0, None)
        Le = np.tile(L[d] * DT, (K, 1))
        Pe = np.tile(Pe_d, (K, 1))
        pk = price_of_day(d)
        c, Aeq, beq, Aub, bub, bd, integ, nk, nz = mp._build(Le, Pe, pk, s, LAM, [])
        r = linprog(c, A_eq=Aeq, b_eq=beq, A_ub=Aub, b_ub=bub, bounds=bd, method="highs")
        if not r.success:
            raise RuntimeError("d=%d" % d)
        G = r.x[0:T]
        ss = s
        H = np.zeros(T)
        for t in range(T):
            gap = L[d][t] * DT - P[d][t] * DT - G[t]
            if gap > 1e-12:
                dd = max(0.0, min(EBAR, ETA * max(0.0, ss - SMIN), gap))
                ss -= dd / ETA
                H[t] = gap - dd
            else:
                room = max(0.0, min(EBAR, (SMAX - ss) / ETA, -gap))
                ss += ETA * room
        Gs[i] = G
        Hs[i] = H
        s = ss
    print(" 计划来源=%s 已生成 (334 天) " % tag, flush=True)
    return Gs, Hs

```

```

def settle(Gs, Hs, price_of_day):
    plan = 0.0
    emg = 0.0
    for i, d in enumerate(DAYS):
        pk = price_of_day(d)
        plan += float((pk * Gs[i]).sum())
        emg += float((5.0 * pk * Hs[i]).sum())
    return plan, emg, plan + emg

print("=== 2x2 反事实交叉结算 ===", flush=True)
G1, H1 = build_plan(lambda d: PRICE1, "附件1 价下优化 (=Q2 计划)")
G4, H4 = build_plan(lambda d: P4[d], "附件4 价下优化 (=Q4-2 计划)")

p1_of = lambda d: PRICE1
p4_of = lambda d: P4[d]
r = {}
r["G1P1"] = settle(G1, H1, p1_of)
r["G1P4"] = settle(G1, H1, p4_of)
r["G4P1"] = settle(G4, H4, p1_of)
r["G4P4"] = settle(G4, H4, p4_of)

print("", flush=True)
print("| 计划来源 | 结算价格 | 计划购电费 | 紧急购电费 | 总成本 | 相对 Q2 |", flush=True)
print("| --- | --- | ---: | ---: | ---: | ---: |", flush=True)
names = {"G1P1": ("Q2 计划(附件1下优化)", "附件1"), "G1P4": ("Q2 计划(附件1下优化)", "附件4"),
         "G4P1": ("Q4 计划(附件4下优化)", "附件1"), "G4P4": ("Q4 计划(附件4下优化)", "附件4")}
base = r["G1P1"][2]
for k in ("G1P1", "G1P4", "G4P1", "G4P4"):
    pl, em, tt = r[k]
    print("| %s | %s | %s | %s | **s** | %s |" % (names[k][0], names[k][1],
        format(pl, ".2f"), format(em, ".2f"), format(tt, ".2f"), format(100 * (tt - base) / base, "+.3f") + "%"), flush=True)

price_eff = r["G1P4"][2] - r["G1P1"][2]
reopt_eff = r["G4P4"][2] - r["G1P4"][2]
print("", flush=True)
print(" 总差 = %s 元" % format(r["G4P4"][2] - r["G1P1"][2], ".2f"), flush=True)
print(" 其中(1)价格环境效应 (同一 Q2 计划只换结算价) = %s 元" % format(price_eff, ".2f"), flush=True)
print(" 其中(2)计划重优化效应 (附件4 价下重新优化) = %s 元" % format(reopt_eff, ".2f"), flush=True)
print(" 校验: (1)+(2) = %s 元" % format(price_eff + reopt_eff, ".2f"), flush=True)

json.dump(dict(cells={k: dict(J_plan=v[0], J_emg=v[1], total=v[2]) for k, v in r.items()},
               price_effect=price_eff, reopt_effect=reopt_eff,
               total_diff=r["G4P4"][2] - r["G1P1"][2]),
          open(OUT + r["q4_counterfactual_2x2.json", "w", encoding="utf-8"), ensure_ascii=False, indent=1)
print("已落盘 q4_counterfactual_2x2.json", flush=True)

```

附录 C 支撑材料文件列表

支撑材料压缩包按问题分组，目录结构与主要内容如下（完整清单与哈希见包内总说明文件和 manifest）。

- Q1 交付/：问题一求解与验证，含求解器、验证器、灵敏度脚本与独立复算报告
- Q2 交付/：问题二求解与验证，含计划层 MILP、紧急购电分解、因果裕度打包与四指定日期表
- Q3 交付/：问题三求解与验证，含全年滚动重跑、物化驱动、对齐检查与对照留档
- Q4 交付/：问题四求解与验证，含确定性序列重算、交叉结算、验证器与结果表
- 结果文件/：result1.xlsx、result2.xlsx、result3.xlsx、result4-2.xlsx、result4-3.xlsx
- AI 工具使用详情.pdf：AI 工具名称、使用环节、提示方式与人工核验说明

全部源程序在提交前做过可运行性检查：脚本自带数据路径与随机种子，按复现说

明顺序执行即可重算论文中的全部数字。